Engineering with Computers (2022) 38 (Suppl 2):S1393–S1425
https://doi.org/10.1007/s00366-020-01247-w

ORIGINAL ARTICLE

A surface mesh deformation method near component intersections for high-fidelity design optimization

Anil Yildirim[1] · **Charles A. Mader**[1] · **Joaquim R. R. A. Martins**[1]

Received: 19 April 2020 / Accepted: 16 December 2020 / Published online: 27 January 2021
© The Author(s), under exclusive licence to Springer-Verlag London Ltd. part of Springer Nature 2021

Abstract
Aerodynamic shape optimization based on computational fluid dynamics (CFD) requires three steps: updating the geometry based on the design variables, updating the CFD surface mesh for the new geometry, and updating the CFD volume mesh based on the new surface mesh. While there are many tools available for the first and third steps, the methods available for the second step are insufficient for geometries that have intersecting components. For these geometries, the CFD surface mesh needs to be updated near component intersections to conform to the component geometries and the updated intersection curves. To address this need, we introduce a method that can deform the CFD surface mesh nodes near component intersections. The method can handle arbitrary design changes for each intersecting component as long as the geometric topology is unchanged. Furthermore, the method is suitable for gradient-based optimization because it smoothly deforms every CFD surface node without introducing topological changes in the CFD surface mesh. In this paper, we detail each step of the proposed method and visualize the range of design changes that can be achieved with this approach. Finally, we use the proposed method in an aerodynamic shape optimization problem to optimize the wing-body intersection of the DLR-F6 configuration. These results demonstrate the effectiveness of the proposed method in a high-fidelity design optimization framework. The method applies to both structured and unstructured CFD meshes and makes it possible to use computer-aided design and conceptual design geometry tools within high-fidelity design optimization.

Keywords Surface mesh deformation · Component intersections · Design optimization · Geometry manipulation · Computational fluid dynamics

1 Introduction

In aircraft design optimization studies, high-fidelity computational fluid dynamics (CFD) simulations are used to evaluate the aerodynamic performance of the given design [1–8]. The CFD mesh needs to be updated as the outer mold line (OML) of the design changes to perform CFD-based shape optimization. Therefore, the design optimization framework requires a robust process that takes the geometric design variables that define the OML as inputs and outputs an updated volume mesh for the CFD solver. There are two main approaches to achieve this: We can either re-create a mesh starting from the new OML or deform an existing mesh to conform to the new OML.

Despite recent advancements in automated meshing software tools [9–11], these tools are still not fast and robust enough to be used in a high-fidelity design optimization framework. Furthermore, topology changes in the computational mesh between design iterations result in numerical noise in the functions of interest, which results in additional difficulties during optimizations [12]. As a result, instead of creating a new computational mesh at each design iteration, we focus on modifying the initial mesh to conform to the updated OML.

The process for creating an updated volume mesh using geometric design variables can be broken down into three main steps: (1) translating geometric design variables into updated surfaces that define the OML of the design; (2) deforming the CFD surface mesh based on the OML; and (3) propagating the displacements of the CFD surface mesh to the CFD volume mesh. In Sects. 1.1–1.3, we summarize the available approaches and recent advancements

✉ Anil Yildirim
anily@umich.edu

[1] Department of Aerospace Engineering, University of Michigan, Ann Arbor, MI 48109, USA

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

for each of these steps. We present the scope of the method we introduce in this paper in Sect. 1.4.

1.1 Geometry

In the first step of this process, the geometry tool generates the updated design OML from the geometric design variables. The review articles by Samareh [13] and Zhang et al. [14] provide summaries of various geometry parametrization approaches used in multidisciplinary design optimization (MDO). Despite the wide range of available geometry parametrization approaches, the free-form deformation (FFD) approach has been the most common geometry manipulation approach used in this field [15, 16].

In the FFD approach, the design changes are represented as deformations of a volume instead of computing the OML. This approach is preferred because of its flexibility and the ease of computing derivatives analytically, which are required for gradient-based optimization. However, this approach does not support the needs of industrial applications [17] and makes it challenging to communicate the optimization results between institutions.

Besides the FFD approach, other geometry parametrization methods in design optimization include computer aided design (CAD) software [18–20] and conceptual design geometry (CDG) tools [21–23]. These tools are the preferred geometry models from an industrial standpoint, and they have many advantages over the FFD approach. Most importantly, these tools represent design intent directly: the geometry parameterization with these tools uses human-readable design variables instead of arbitrary deformations of the FFD control points. As a result, designers can apply their intuition during an optimization process, and the results of optimizations can easily be communicated between institutions and optimization frameworks.

While the use of CAD and CDG tools in MDO has been limited [24–26], recent advancements in these tools have removed some of the technical barriers causing this. Dannenhoffer and Haimes [27] modified the Engineering Sketch Pad (ESP) [19] to provide derivatives of the geometry model with respect to the design variables. To compute the derivatives, they used a mixture of algorithmic differentiation (AD) and finite differences (FD). Similarly, Banović et al. [28] performed AD of the Open CASCADE Technology, and Agarwal et al. [29] developed a parametric design velocity computation technique using adjoint methods. These advancements provide the foundations for using these geometry tools within the context of gradient-based optimization. However, additional tools to deform the surface meshes using these geometry models are required, which we discuss in the next section.

1.2 Surface mesh deformation

The second step in this process is to update the CFD surface mesh to conform to the updated OML, and several approaches exist for this purpose. For the FFD approach, the surface mesh nodes can be directly embedded in the FFD volume [15]. This is equivalent to *projecting* these nodes to the three-dimensional volume. In this context, a projection can be viewed as finding the parametric coordinates of the projection point in the FFD volume, where the Cartesian coordinates of the projection point are within some tolerance relative to the original point. Once every surface CFD node is embedded in the FFD volume, these points can be tracked throughout the optimization process. Their Cartesian coordinates can be re-evaluated using the parametric coordinates of the projections every time the FFD volume is deformed to change the design geometry.

For CAD and CDG tools, where the geometry is defined as a continuous surface, a similar process can be followed. For these cases, the surface mesh nodes can again be *projected* to the continuous surface geometry. In this context, a projection is equivalent to finding the parametric coordinates of the projection point on the continuous surface that is closest to the surface mesh node in Cartesian coordinates. Once these parametric coordinates are found, the surface mesh nodes can be updated by forming rigid links between these projection points, or the Cartesian coordinates of the projected points can be used directly to update the surface mesh nodes [30, 31]. Furthermore, with these geometry tools, the surface mesh generation process itself can be automated. If the parametric coordinate information of the initial surface mesh is saved, this can be used later to update the surface mesh without projecting the points to the continuous surfaces [18, 32].

Using these approaches, deforming the surface nodes of a component that does not intersect with any other component is straightforward. However, regardless of the geometry tool, no established method exists to robustly handle design changes near intersecting components. Once the design of intersecting components changes, the surface mesh nodes on both components must be displaced to conform to the new component intersection [12]. This is because CFD solvers require airtight computational meshes that conform to the design OML.

There are several approaches to address this problem. Kim and Nakahashi [33] developed a surface mesh deformation method near junctions using the geodesic displacements of the intersection nodes and a mesh deformation approach based on a spring analogy. Koc et al. [34] used this method to perform one of the earliest examples of design optimization that includes component intersections. However, this method uses a surface reconstruction

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

step, and as a result, the deformed surface mesh nodes are not guaranteed to lie on the design OML. Furthermore, the method cannot guarantee to preserve features such as a blunt trailing edge because the surface mesh is deformed only with the displacement of the intersection nodes. There are no corrections for the displacements of the curves that define these features. Brezillon and Dwight [35] performed optimization of a wing-fuselage intersection, where they deformed the CAD control points using FFD volumes. In this work, after obtaining an updated CAD model, they re-generated the computational mesh at every iteration. Furthermore, they used FD to obtain the derivatives of mesh points with respect to the design variables, which required generating as many meshes as the number of design variables at each design iteration. Because each mesh generation took on the order of a minute to complete, this method is impractical for gradient-based optimization involving hundreds of design variables. Xu et al. [36] performed a CAD-based optimization of a wing-body junction, where the surface mesh nodes are deformed in parametric space to conform to the updated intersection curve at each iteration. This approach limited the design changes that can be achieved because the mesh nodes cannot move across surface patches that define the CAD geometry. Mykhaskiv et al. [37] used the method developed by Xu et al. [36] and performed design optimization of a wing-body configuration using a differentiated CAD kernel. All of the approaches described so far were limited in terms of the range of design changes allowed.

Secco et al. [38] developed the most flexible and advanced approach in this field using overset meshes and later used this method to optimize the design of a strut-braced wing [39]. In this approach, the surface collar mesh that conforms to the intersection of the two components is re-computed at every design iteration. Then, the surface nodes of the collar mesh are used to deform the volume nodes of the collar mesh, while the remaining volume cells, consisting of component and background meshes, are deformed with respect to the changes in these surfaces. As a result, this method is much more flexible than previously proposed approaches: the users only need to maintain a consistent geometric topology at each design iteration. In this context, a consistent geometric topology means that the components that intersect each other with the original design also intersect when the design is updated. The components that do not initially intersect do not form a new intersection as the design is updated. However, even minimal design changes can change the overset-mesh topology, while the geometric topology is unchanged. This change in the overset-mesh topology can cause the optimizer to have problems to converge during the final stages of an optimization because of the noise introduced by the changing mesh topology [12], which is the main disadvantage of the method. Furthermore, the method is demanding to set up and involves many tunable parameters that might not be optimal for a wide range of applications.

1.3 Volume mesh deformation

In the third step of this process, the surface mesh nodes' changes are propagated to the volume mesh nodes. Once the coordinates of the nodes that define the surface mesh are updated, many well-established mesh deformation methods can be used to propagate the changes in the surface nodes to the volume nodes.

Methods based on radial basis functions (RBF) [40, 41] provide great robustness. However, these methods require solving a linear system whose size is equal to the number of boundary nodes used to prescribe displacements.

Besides RBF based methods, there are several approaches based on an inverse-distance weighing [42]. While these methods only contain explicit computations, the number of computations required to update a large mesh can be limiting. To address this, Luke et al. [43] proposed an approximation of this approach and demonstrated that the method's performance could be greatly improved with a small sacrifice in accuracy. Secco et al. [44] detail efficient mesh generation and deformation methods for MDO, and they introduce the IDWarp package, which is a volume mesh deformation tool based on the method developed by Luke et al. [43]. IDWarp is available under an open-source license, and it is the volume mesh deformation tool we use herein.[1]

1.4 Scope of the proposed method

There are many well-established and robust approaches for the first and third steps in this process for obtaining an updated volume mesh starting from design variables [45]. However, progress in tools for the second step has been limited, and there is no standard approach for handling design changes near component intersections. Each method listed in Sect. 1.2 has a fundamental limitation that prevents its extensive use in high-fidelity MDO frameworks. Furthermore, these tools' limitations prevent the use of component-based design tools such as CAD and CDG in high-fidelity design optimization.

In this paper, we present a new method to update surface meshes near component intersections. The method applies to both structured and unstructured CFD meshes and enables component-based design changes near component intersections. Compared to existing approaches, the proposed method has several advantages. First, the method is easy to use because it requires minimal setup by the user with no need for fine-tuning for each configuration.

[1] https://github.com/mdolab/idwarp, Accessed December 2020.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Second, the method smoothly deforms every surface mesh node near intersections. Therefore the mesh topology and interpolation weights are preserved when an overset mesh is used. This results in a numerically smooth design space and avoids the numerical noise issues that arise with methods that modify the overset mesh. With these improvements, the proposed method enables the use of component-based geometry tools within high-fidelity design optimization.

Third, the surface mesh deformation algorithm is analytically differentiated. If the geometry tool also provides analytical derivatives, the method can be used to compute analytic derivatives of surface node coordinates with respect to the geometric design variables. On the other hand, if the geometry tool does not provide analytical derivatives, users can use FD with the geometry tool to obtain the derivatives of points on the surface with respect to shape design variables. This information can then be used together with the analytically differentiated surface mesh deformation method to obtain the derivatives of surface mesh coordinates with respect to shape design variables.

The remainder of the paper is organized as follows. In Sect. 2, we detail the steps required for the proposed method. Because the method contains multiple steps, we summarize the overall method in Sect. 2.10. In Sect. 3, we demonstrate the capabilities of the method using two example configurations, and in Sect. 4 we apply the method to a CFD-based aerodynamic shape optimization case to demonstrate the effectiveness of the method in an optimization framework. Finally, we present the conclusions of this study in Sect. 5.

2 Surface mesh deformation method

The goal of this method to obtain the updated CFD surface mesh node coordinates $\mathbf{x}_S$, given the updated design OML. This can be written as

$$\mathbf{x}_S = \mathbf{x}_{S_0} + \Delta\mathbf{x}_{S_0}, \tag{1}$$

where $\mathbf{x}_{S_0}$ represents the surface mesh coordinates computed with the initial OML S_0, and $\Delta\mathbf{x}_{S_0}$ is the change in surface coordinates after a design change. We focus on a single-step approach, where the surface mesh node coordinates for each design are obtained by computing the displacement $\Delta\mathbf{x}_{S_0}$, starting from the initial coordinates $\mathbf{x}_{S_0}$. The initial surface mesh coordinates $\mathbf{x}_{S_0}$ are computed by an external meshing tool as a pre-processing step, where the surface mesh represents the surface of the aerodynamic problem. These surface mesh nodes should be sufficiently close to the actual design OML so that the distances of these points to the CAD surface are within the meshing tolerance.

The proposed method is implemented using the pySurf module developed by Secco et al. [38], which contains a wide range of geometric operations defined on triangulated surfaces and piecewise linear curves. Furthermore, the operations in this module are differentiated using AD, and we use these differentiated routines to obtain the derivatives of the resulting surface mesh coordinates with respect to the design variables. In the following sections, we detail each step of the proposed surface mesh deformation approach and summarize the method in Sect. 2.10.

2.1 Example wing-body configuration

To visualize each step of the proposed method, we use the DLR-F6 wing-body configuration as an example case [46]. The initial geometry and the surface mesh are shown in Fig. 1a and b, respectively. This configuration has a wing-fuselage intersection, which is a common feature in aircraft configurations. As the design OML is modified during an optimization procedure, the surface mesh nodes need to be modified to conform to the updated geometry while preserving the mesh resolution near the intersection and geometry features.

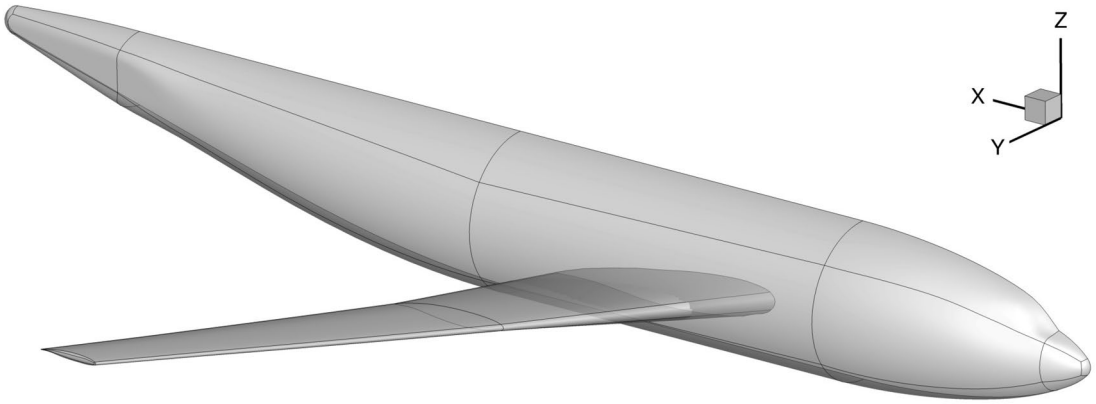
2.2 Design changes for each component

In this work, we use the FFD approach to parametrize the geometry [15]. Each component has its own FFD volume that controls the design of that component independently from other components. The FFD volumes for the wing-body configuration, along with the component geometries for the wing and the fuselage are shown in Fig. 2a and b, respectively. Using these FFD volumes, we can parametrize the geometry of each component separately. For example, we can move every FFD control point of the wing to adjust the wing's position with respect to the fuselage.

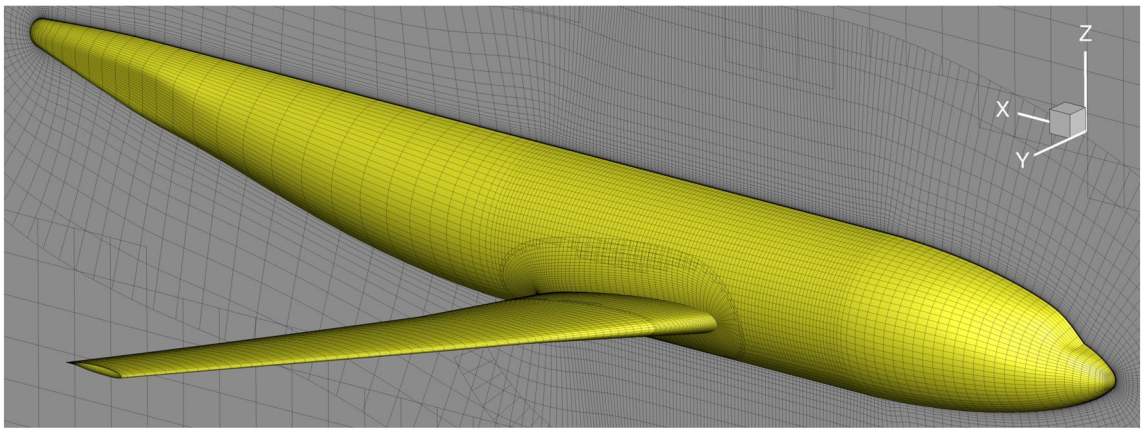
To demonstrate each step of the proposed surface mesh deformation method, we define a local geometric design change on the fuselage near the wing-fuselage intersection, where we control 16 FFD points (Fig. 3a). These FFD control points are only allowed to move in the direction normal to the fuselage surface, and we move these points away from the fuselage center. This is identical to the parametrization used by Secco et al. [47]. This design change perturbs a local area of the fuselage outward, effectively creating a wing fairing, as shown in Fig. 3b. This is a large design change that is not practical; however, it is a good test case to demonstrate the robustness of the surface mesh deformation method.

Besides the FFD volumes, we create a triangulated surface mesh for each component and track these triangulated mesh nodes using each component's respective FFD volumes. To embed the triangulated surface mesh nodes in the respective FFD volumes, we find the parametric coordinates (u, v, w) of the points in the FFD volumes that are closest to the Cartesian coordinates of the triangulated surface mesh

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



(a)



(b)

Fig. 1 Initial geometry and surface mesh of the DLR-F6 wing-body configuration. **a** DLR-F6 wing-body configuration geometry. The geometry of each component is defined independently and the root section of the wing is visible behind the fuselage. **b** Surface mesh of the initial design. Only the active cells in the overset mesh are visualized. The aircraft surface mesh is shown in yellow and the symmetry plane is shown in gray. While we use a structured mesh in this example, the method is also applicable to unstructured grids (color figure online)

nodes. Then we track these parametric coordinates as the design is changed by deforming the FFD volumes [15]. This operation can be viewed as a *projection* of the triangulated surface mesh nodes onto the FFD volumes. The triangulated mesh nodes for each component only get displaced by that component's FFD deformations. We use these triangulated surfaces as the reference component surface when performing several operations, including finding the intersection curve between the two components.

As we describe in the following sections, during the CFD surface mesh deformation process, we use these triangulated meshes with a range of geometric operations that are defined on discrete geometric entities. While all of these operations can also be performed using continuous definitions of these component surfaces (which are available in CAD models [48]), we focus on operations on discrete geometries. We prefer this approach because these operations using discrete surfaces are more robust, and we have analytically differentiated versions available in the pySurf module [12].

The wing and fuselage triangulated surface meshes for the DLR-F6 configuration are shown in Fig. 4a and b, respectively. These triangulated surface meshes provide the surface information for each component, including the regions where the CFD surface mesh is not defined, such as the wing root and the region on the fuselage that is covered by the wing intersection. The fuselage triangulated surface mesh contains 109,127 nodes, and 217,728 elements, while the wing triangulated surface mesh contains 93,981 nodes and 187,583 elements.

Along with the triangulated surface mesh nodes, we also need to track the CFD surface mesh nodes with the FFD volumes. The proposed surface mesh deformation algorithm treats the surface mesh nodes as a *cloud of points*, that do not have any connectivity or component information. Because

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

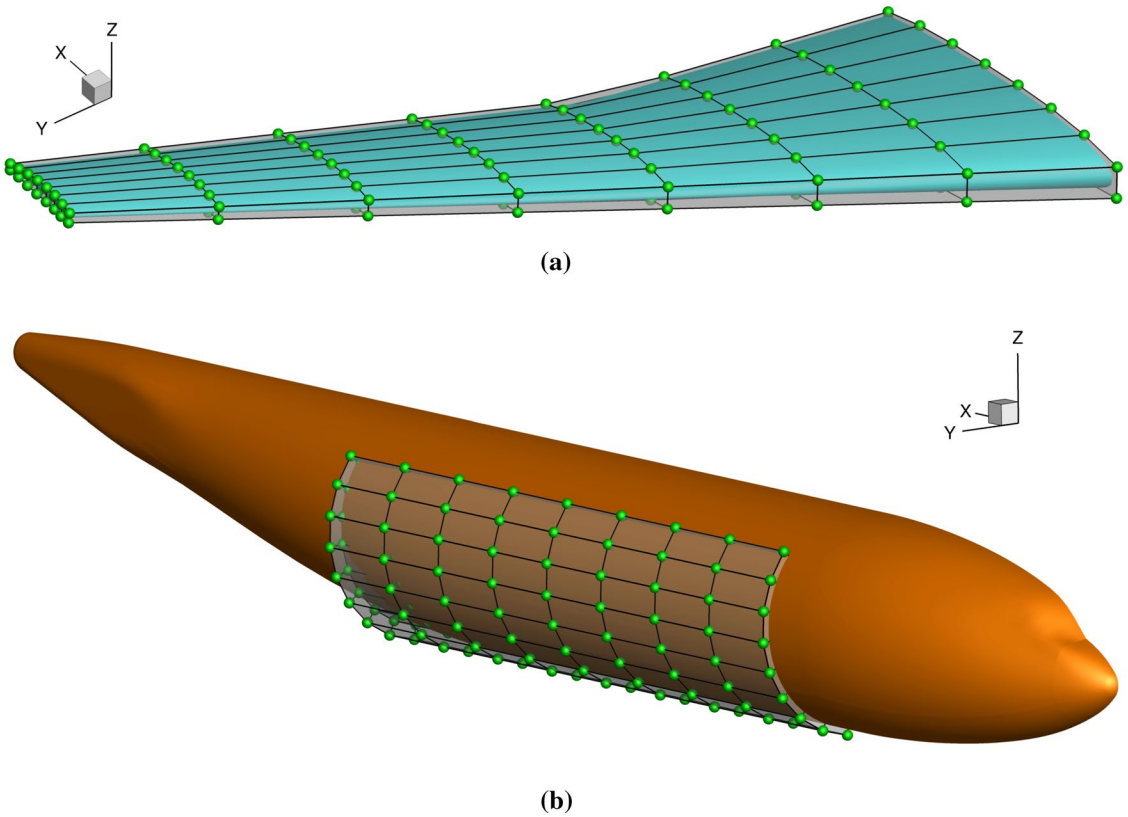


Fig. 2 FFD volumes defined for each component. In this work, we displace the FFD control points to deform the volume inside the FFD boxes to modify the component shape. **a** FFD volume that parametrizes the wing design. The wing surface is shown in blue, and the FFD control points are shown in green. **b** FFD volume that parametrizes the fuselage. The fuselage surface is shown in orange and the FFD control points are shown in green (color figure online)

of this approach, the method can be used with structured and unstructured CFD meshes since the only information required for the method is the coordinates of the surface mesh nodes. However, because there is no connectivity or component information for any surface node, we need to determine which surface mesh node should be mapped to which component's FFD volume. The user can provide this information since the node-to-component mappings can be obtained from the meshing tool used to create the CFD surface mesh. However, this increases the user effort required to set up and use the proposed surface mesh deformation method. Therefore, we avoid this requirement by automatically determining the CFD node to component mapping.

In the areas away from the wing-fuselage intersection, the node-to-component mapping is straightforward: if the point is inside one FFD volume but outside the others, the point is tracked by this FFD volume. For example, a CFD surface node near the wingtip is outside the fuselage FFD volume and inside the wing FFD volume for the DLR-F6 configuration. Therefore, we can simply embed this point in the wing FFD volume.

However, the CFD surface points near the wing-fuselage intersection can be inside both of the FFD volumes. To determine which FFD volume we need to use to track these points, we simply project these points to both of the triangulated surfaces and embed the point to the component's FFD volume that yields a smaller projection distance. For example, a CFD surface point on the fuselage but near the intersection would be embedded in the fuselage FFD volume since the projection to the fuselage surface is guaranteed to yield a smaller projection distance than the projection to the wing surface.

For the proposed method, the mapping of points on the intersection curve to the components does not make any difference in the resulting surface mesh, as explained in the following sections. Therefore, the CFD surface points that are on the intersection curve can be embedded to either FFD volume, and this can be similarly determined by the distances to each triangulated surface mesh, where the projection distances for these points are on the order of the meshing tolerance. Finally, if a point is not inside any FFD volume, it is not embedded in any FFD, and therefore it is not affected by the FFD volume deformations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

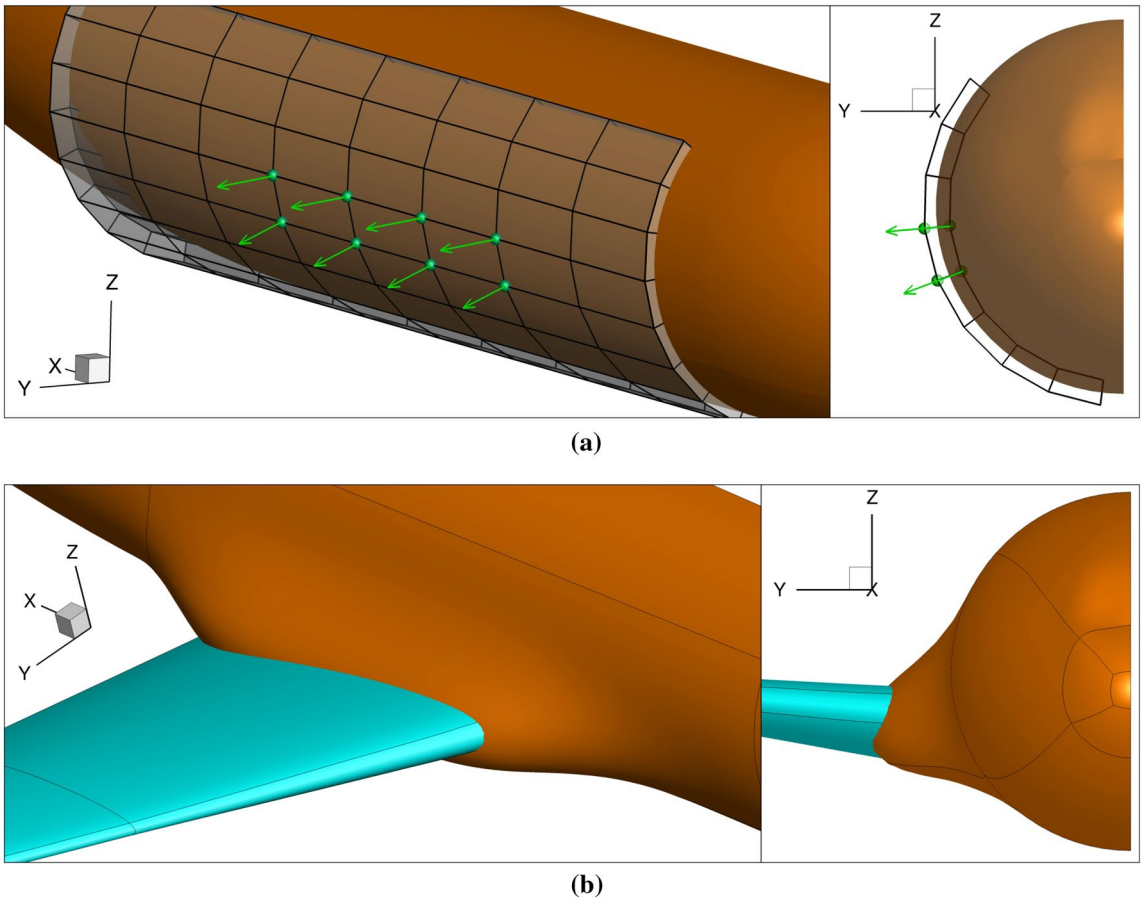


Fig. 3 A fuselage design change that demonstrates the surface mesh deformation algorithm. **a** The 16 FFD control points (shown in green) move radially away from the fuselage center, creating a wing fairing around the cylindrical fuselage section. We maintain C_1 continuity with the rest of the fuselage geometry, which is not modified, by keeping at least two control points stationary in each direction [47]. **b** The geometry resulting from the FFD control point movements is a large design change that demonstrates the robustness of the method (color figure online)

As the design of each component is modified, we denote the surface mesh displacements caused by the FFD volume deformations as $\Delta \mathbf{x}_{\text{FFD}}$. If the geometry does not have any intersecting components, the updated surface coordinates can be obtained by evaluating the Cartesian coordinates of the embedded points using the parametric coordinates. However, this would yield an invalid CFD surface mesh near the component intersections. While we compute this displacement for every surface node, this displacement alone is not sufficient to obtain a valid CFD surface mesh when the component intersections change between design changes. To visualize this, we deform all CFD surface nodes of the DLR-F6 configuration, shown in Fig. 1b, using $\Delta \mathbf{x}_{\text{FFD}}$ alone. We use the example design change introduced in Fig. 3b, and the resulting surface mesh is shown in Fig. 5. This example demonstrates that while the FFD deformations are sufficient to obtain a valid surface mesh away from component intersections, we need to include an additional treatment for the surface mesh nodes near the intersection.

While we use the FFD approach in this work, the method applies to other geometry parametrization approaches, including CAD and CDG tools. To use any geometry tool with the proposed method, we need to track the CFD and triangulated surface mesh nodes with that tool. With the FFD approach, this is easy because we can embed these points in the FFD volumes. This process is not straightforward with CAD and CDG tools, where a continuous surface for each component is defined, but there is no parametric volume that can be used to track the surface points.

The most straightforward approach for this case is to project each point onto the continuous component surface in the CAD or CDG tool and track the parametric (u, v) coordinates of these projections as the design changes. This is analogous to embedding the points in the FFD

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

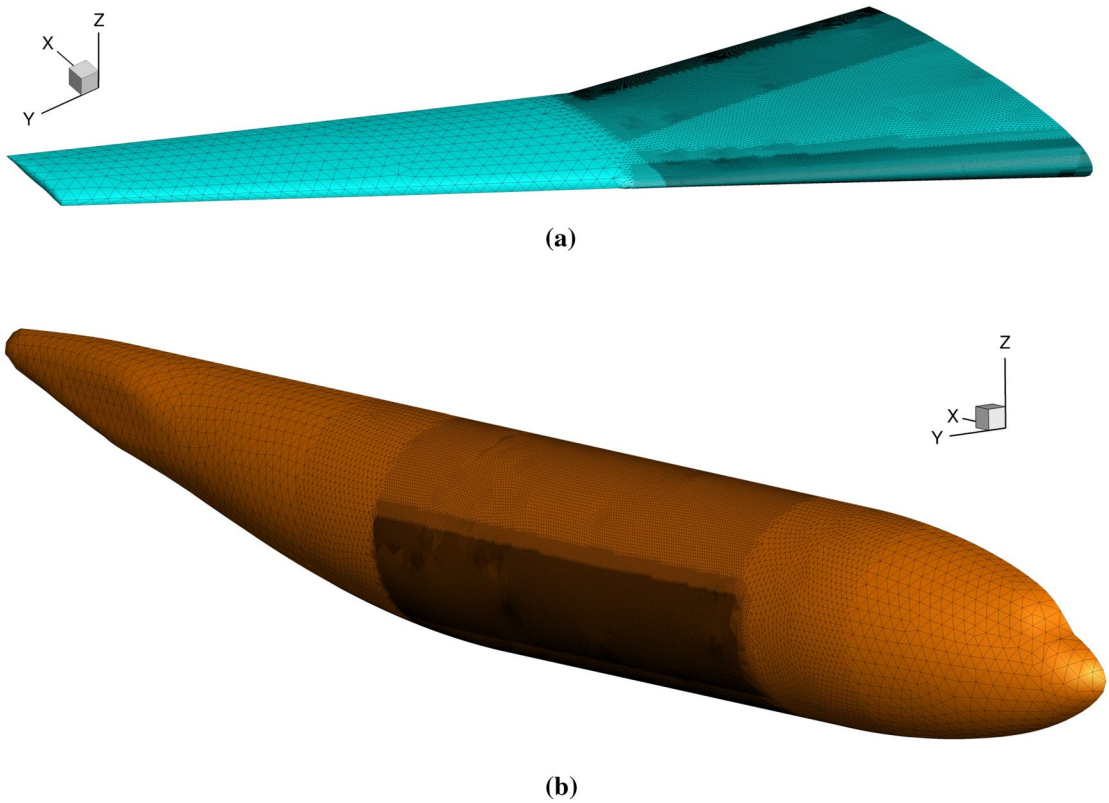


Fig. 4 Triangulated surface meshes for the wing and fuselage that are used to define the component surfaces. Both of these meshes contain an increased resolution near the component intersection to define an accurate surface description when deforming the CFD surface nodes around the intersection curve. These triangulated surface meshes only get deformed by the FFD volumes of the respective components. **a** The triangulated surface mesh of the wing includes the wing-root section, which is inside the fuselage geometry in the combined wing-body configuration. **b** The triangulated surface mesh of the fuselage includes the region on the fuselage surface that is covered by the wing intersection in the combined wing-body configuration

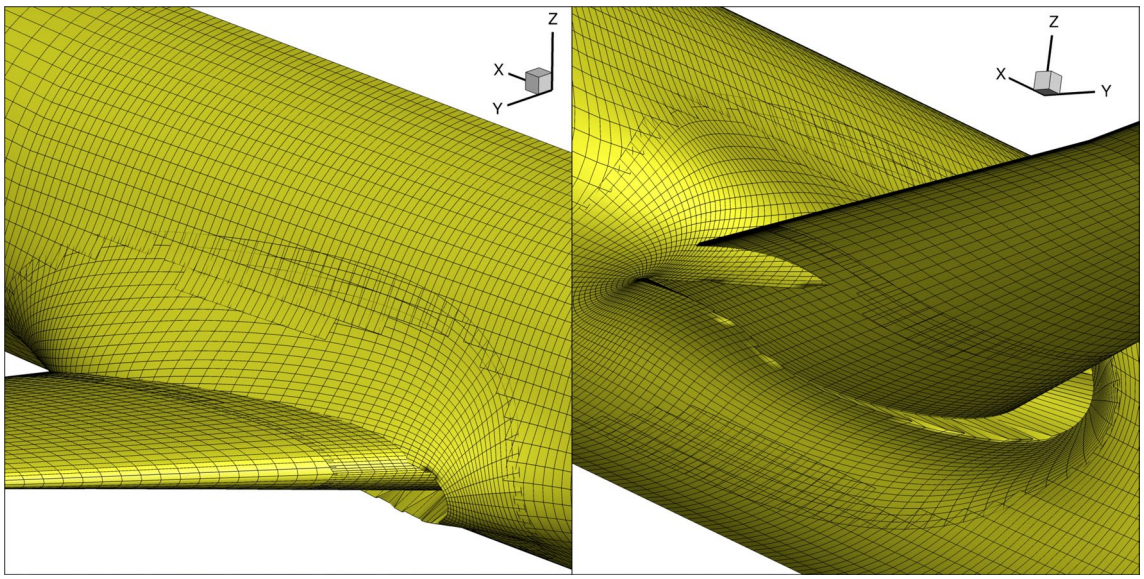


Fig. 5 Resulting surface mesh with only $\Delta \mathbf{x}_{\text{FFD}}$ using the design change introduced in Fig. 3b, and without any intersection treatment. The figure on the left shows the wing leading edge viewed from the top side of the aircraft, while the figure on the right shows the wing trailing edge viewed from the bottom side. This is not a valid CFD surface mesh because the surface mesh intersects itself, and there are many invalid cells near the wing-fuselage intersection

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

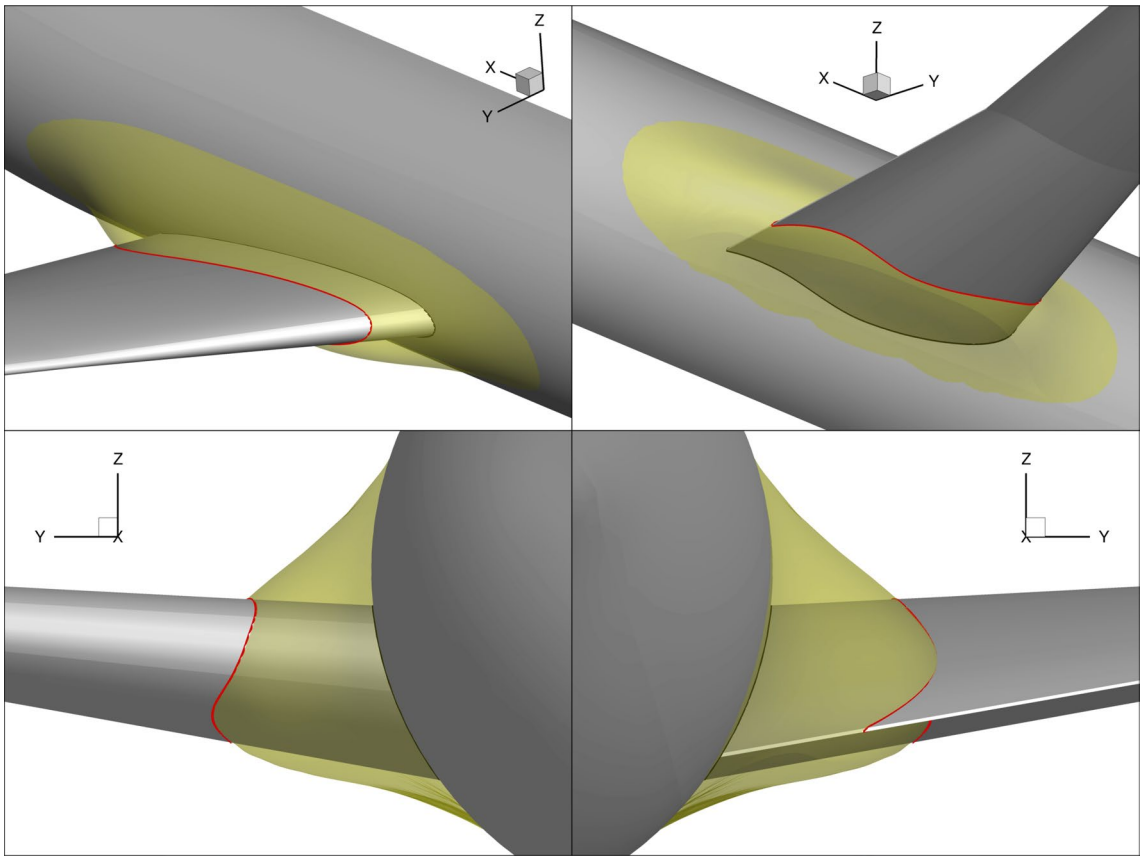


Fig. 6 Intersection curves with the baseline and the modified design. The perturbation to the fuselage geometry is shown in translucent yellow. The initial intersection curve is shown in black, while the updated intersection curve is shown in red. The top two figures show the three-dimensional views of the leading edge and trailing edge. The bottom left figure shows the front view, and the bottom right figure shows the rear view of the wing-body intersection (color figure online)

volumes; when we have continuous component surfaces, we can project the points onto these continuous surfaces instead of projecting them onto the FFD volumes.

Beyond this simple approach, there are many available methods in the literature that can be used to track surface points using continuous surface geometries [31, 32], and any of these can be used to get the desired result. These methods primarily define a way to track these surface points on each component separately. The tracking in these methods can be combined with the surface mesh deformation approach developed in this paper.

2.3 Computation of the intersection curve

The proposed surface mesh deformation algorithm is built around the idea of propagating the displacements of curves of the geometry to the surface mesh nodes as the design is changed. The first step in this approach is the computation of the displacement of the intersection curve. To do this, we compute the intersection curve with the baseline design once and compute the updated intersection curve as the design is updated. We use the triangulated surface meshes for this computation, and Secco [12] provides the details of the intersection curve computation. Figure 6 shows the baseline and updated intersection curves for the example design change introduced in Fig. 3b.

The intersection curve computed with the triangulated surface meshes are piecewise linear curves that are defined with line elements. Each line element is defined by the two endpoint coordinates, and we compute the displacement of the curve as the design is changed using the displacement of the nodes that are used to define the curve. To define the displacements with this approach, we need to preserve the topology of the curve and the number of nodes used to determine the curve at each design iteration.

In this context, preserving the curve topology means that the nodes on a segment of the curve are mapped to the updated nodes on the same segment so that the displacements that are computed on these nodes are consistent between design iterations. For the example configuration introduced in Sect. 2.1, the intersection curve between the wing and the fuselage is a closed curve around the wing

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

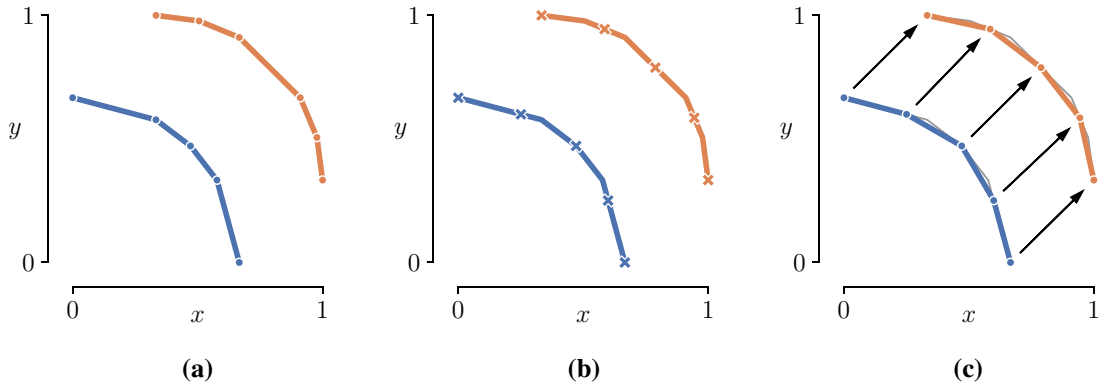


Fig. 7 Re-meshing procedure for the curve segments on the baseline (blue) and updated (orange) designs. This step ensures that each segment of the re-meshed intersection curve has the same number of nodes as the design is modified. With this approach, we compute the displacements of each node on curve segments as the design is changed. **a** Original curves for the baseline (blue) and updated (orange) designs. Nodes that define the curves are shown with dots. **b** Re-sampled points on the original curves. These points are not guaranteed to be colocated with the original nodes. **c** Re-meshed curves for both designs. Original curves are shown in gray. The displacement of each node is shown with an arrow (color figure online)

geometry. For this case, preserving the curve topology means that the nodes that define the portion of the intersection curve on the upper wing surface are mapped to the nodes in the same location after the design is updated.

To preserve the curve topology, we use features on the intersected components to split up the intersection curve into segments. We create a segment between each geometry feature, and with this approach, the displacements computed for the nodes on the intersection curve are done with the correct location of the updated nodes after a design change. For the example configuration, the features we use for this step are the leading edge curve and the upper and lower trailing edge curves of the wing. As a result, the intersection curve is split into three segments: upper wing, lower wing, and wing trailing edge. With this approach, the nodes that define the segment on the trailing edge with the baseline design are mapped to the updated nodes on the trailing edge. The displacement of these nodes is computed in a consistent way that preserves the curve topology.

After splitting up the intersection curve into segments, we also need to maintain the same number of nodes on each segment between design changes to compute the displacements for each segment. This is done by *re-meshing* every curve segment. This step is required because the count and spacing of nodes on each curve segment can change as the design is changed, as shown in Fig. 7a. During this step, we re-sample nodes on the original curve segments using a constant arc-length spacing, as shown in Fig. 7b. We always use the same number of nodes that is on the original curve segment of the baseline design as the design is changed. While this approach modifies the original intersection curve computed using the triangulated surfaces, these triangulated meshes are much finer than the CFD surface mesh. Therefore, the errors introduced in this step are negligible compared to the CFD surface mesh resolution. Maintaining the node counts and spacings enables the computation of displacements for every node between the baseline and updated designs, shown with arrows in Fig. 7c.

We define the re-meshed intersection curve with the baseline design as $\mathcal{C}_{\mathrm{I}_0}$ and the re-meshed curve with the updated design as $\mathcal{C}_{\mathrm{I}}$. Using these definitions, we define the displacement of the intersection curve as

$$\Delta\mathcal{C}_{\mathrm{I}_0} = \mathcal{C}_{\mathrm{I}} - \mathcal{C}_{\mathrm{I}_0}, \tag{2}$$

which represents the displacements of the intersection curve at each design iteration relative to the baseline design.

2.4 Computation of feature curves

In addition to recomputing and re-meshing the intersection curve, we do the same for all *feature curves* of interest at each design iteration. These feature curves include the edges of the geometry where the component surfaces are C_0 continuous, e.g. for a wing with a blunt trailing edge; these edges consist of the upper and lower trailing edges. On top of these, the feature curves can also include any other curve of interest, such as the leading edge curve of a wing. For this case, even though the surface of the wing can be C_1 continuous near the leading edge, the inclusion of this curve maintains the position of the surface mesh nodes near the leading edge curve with respect to this curve. The set of feature curves used in the algorithm is selected by the user and can include any curve in the geometry. These curves are included in the algorithm to preserve the mesh topology

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

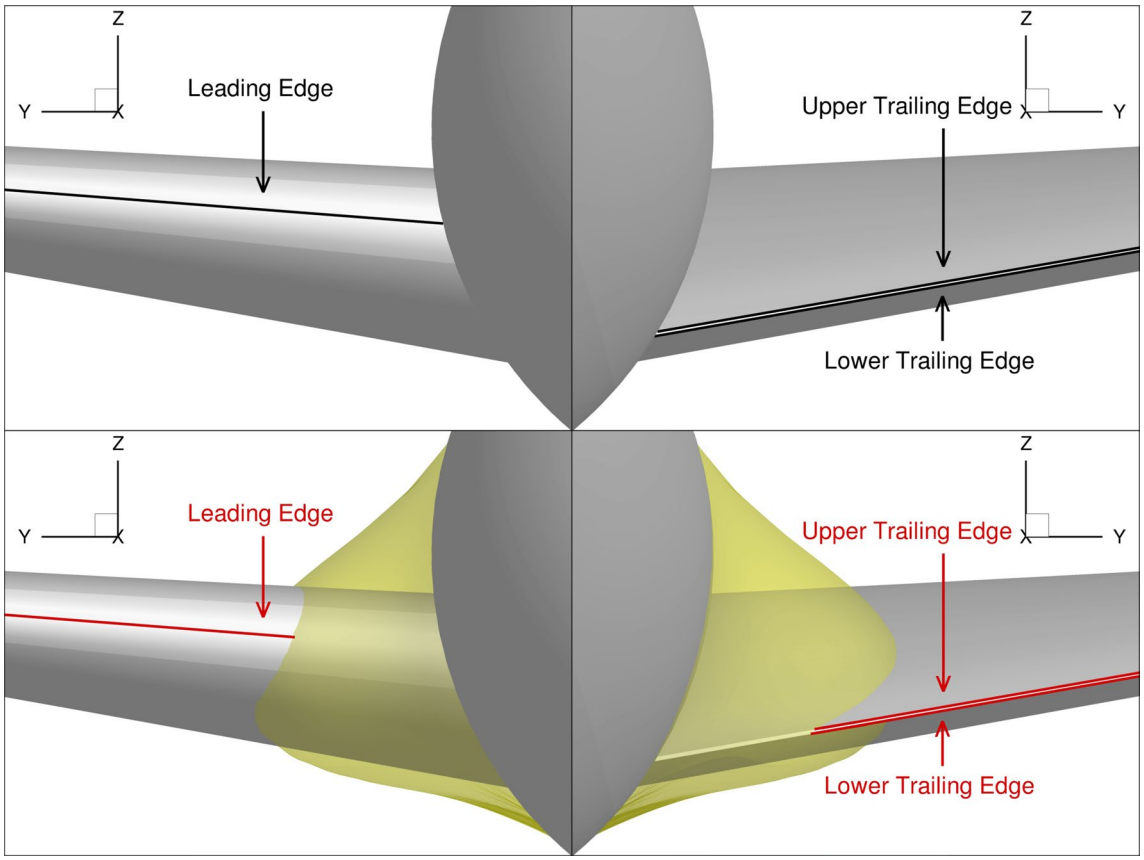


Fig. 8 Wing geometry feature curves that are outside the fuselage geometry. The figures on the top show the baseline design, where the feature curves are plotted in black. The bottom figures show the updated design, where the updated feature curves are plotted in red. The figures on the left show the front view of the intersection region, while the figures on the right show the rear view. The design change in the fuselage is shown in translucent yellow (color figure online)

around important geometry features, as we describe in the following sections.

During the setup phase of the algorithm, we compute the initial feature curves as we did for the intersection curve. Then, we re-mesh these intersection curves using the same number of line elements. After the setup phase, we recompute these feature curves every time the design is updated and again re-mesh them using the same number of line elements as the original feature curves. Using this approach, we ensure that the number of line elements used to define each feature curve does not change with the design changes. We can compute the displacement of the feature curves as

$$\Delta \mathcal{C}_{\mathrm{F}_0} = \mathcal{C}_{\mathrm{F}} - \mathcal{C}_{\mathrm{F}_0}, \tag{3}$$

where $\mathcal{C}_{\mathrm{F}_0}$ represents the union of the feature curves computed with the initial design and $\mathcal{C}_{\mathrm{F}}$ represents the union of feature curves computed after a design change.

As the design is changed, we need to compute these feature curves in a consistent way at each design iteration. For example, if the feature curve is defined to start from the intersection curve, then the updated feature curve should account for the updated location of the intersection curve. This can be seen in Fig. 8 using the example design change we consider herein. When the fuselage is perturbed outboard, the intersection between the wing and the fuselage also moves outboard. As a result, the feature curves need to be re-meshed, starting from this updated location of the intersection curve.

Similarly, this re-meshing is also done for the portion of the feature curves that are on the other side of the component intersection, i.e., inside the fuselage in our example (see Fig. 9). This ensures that the CFD surface cells that are left inside the fuselage are also displaced in a consistent way. While these cells are not active in the overset mesh [49], they also need to be deformed consistently to obtain a consistent volume mesh.

2.5 Curve-based surface mesh deformation

The most important step of the proposed surface mesh deformation algorithm is the inverse-distance weighted

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

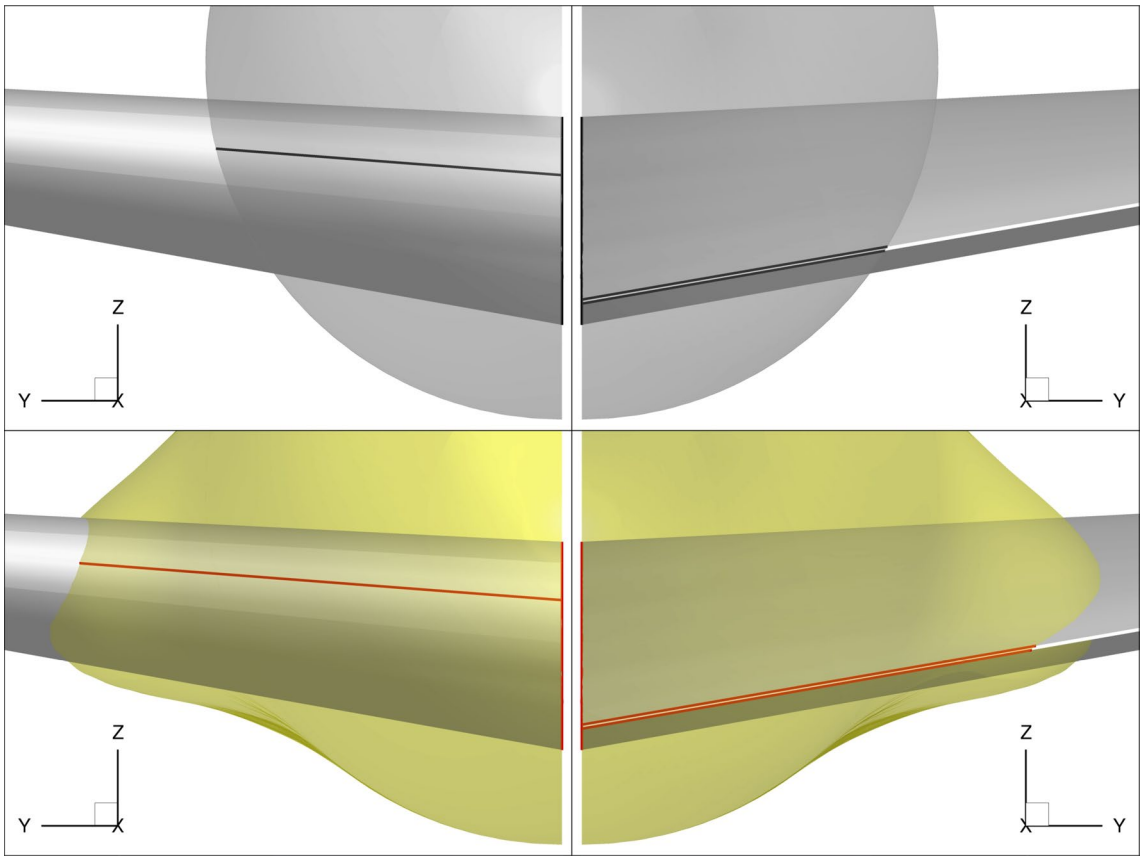


Fig. 9 Wing geometry feature curves that are inside the fuselage geometry. The figures on the top show the baseline design, where the feature curves are plotted in black. The bottom figures show the updated design, where the updated feature curves are plotted in red. The figures on the left show the front view of the intersection region, while the figures on the right show the rear view. The design change in the fuselage is shown translucent yellow. In addition to the leading and trailing edge curves, we also compute the curve that defines the wing root (color figure online)

deformation of the CFD surface nodes using the intersection and feature curves' displacements. The goal of this step is to deform the surface mesh nodes so that the nodes on the intersection and feature curves follow the displacement of these curves exactly. The nearby nodes are deformed in a consistent way to preserve the mesh topology.

To deform the surface mesh nodes near intersections, we use the deformations of both the intersection and feature curves. These are piecewise linear curves that are defined by line elements. We refer to the union of these curves as $\mathcal{C}$. The curves computed with the initial design are denoted as $\mathcal{C}_{S_0}$, and the displacements defined on these curves after a design change are denoted as $\Delta\mathcal{C}_{S_0}$. Finally, we refer to the point coordinates that define the line elements used to form these curves as $\mathbf{p}$. Similarly, the coordinates computed with the initial design are denoted as $\mathbf{p}_{S_0}$ and the displacements after a design change are denoted as $\Delta\mathbf{p}_{S_0}$.

Inverse distance-based mesh deformation approaches have been used to update the CFD volume meshes for a given displacement field on the CFD surface meshes [42–44]. Shepard [50] presented one of the first studies that use this approach to interpolate irregularly spaced data. While many improved versions are available, the simplest form of an inverse distance based deformation approach can be written as

$$\Delta\mathbf{x}_{\text{deform},i} = \frac{\sum_j w_j(\mathbf{x}_{S_0,i})\Delta\mathbf{p}_{S_0,j}}{\sum_j w_j(\mathbf{x}_{S_0,i})}, \tag{4}$$

where $\Delta\mathbf{x}_{\text{deform},i}$ is the displacement evaluated at the ith surface mesh node with coordinates $\mathbf{x}_{S_0,i}$, given the displacements of the points $\Delta\mathbf{p}_{S_0,j}$ that define $\mathcal{C}_{S_0}$. The two summations include a weighting function $w_j(\mathbf{x}_{S_0,i})$ that is defined as

$$w_j(\mathbf{x}_{S_0,i}) = \frac{1}{\left|\left|\mathbf{x}_{S_0,i} - \mathbf{p}_{S_0,j}\right|\right|_2^m}, \tag{5}$$

where, $\left|\left|\mathbf{x}_{S_0,i} - \mathbf{p}_{S_0,j}\right|\right|_2$ is the Euclidean distance between the evaluation coordinate $\mathbf{x}_{S_0,i}$ and the jth point used to define $\mathcal{C}_{S_0}$. In this formulation, this distance is raised to the power

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

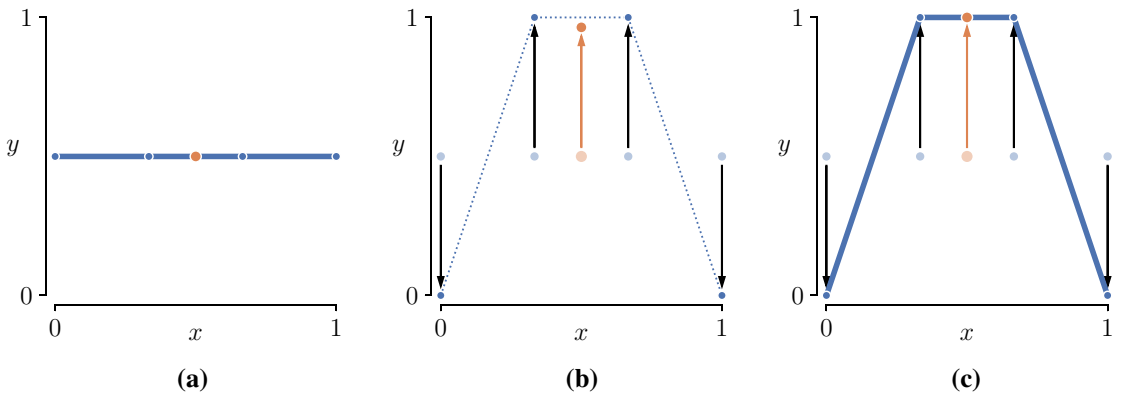


Fig. 10 Canonical example demonstrating the shortcomings of the point-based deformation approach. There are four points in the initial design that define a piecewise linear curve with three line elements (blue in **a**). We want to deform the orange node located on this curve using the displacements of the curve. With a point-based deformation approach, the orange node does not follow the line segment that it was originally located on **b**. However, with a curve-based deformation approach, we can deform the orange point to directly follow the displacement of the line element it is located on **c**. **a** Initial node positions. **b** Updated positions using a point-based deformation approach (4). **c** Updated position using a curve-based deformation approach (6) (color figure online)

m, usually set to 3. This weighting function can be modified to include additional terms, such as the inverse of the Euclidean distance raised to additional powers; however, we use $m = 3$ in this work for simplicity.

The inverse-distance based deformation approaches define a displacement field, given the displacement of a point set. In the context of these approaches, the *seeds* are the locations where the displacements are prescribed, and the *evaluation points* are the locations where the displacement field is evaluated. For example, in a volume mesh deformation approach, where the surface mesh nodes' displacements are propagated to the volume mesh, the surface mesh nodes are the seeds, and the remaining volume mesh nodes are the evaluation points.

The deformation approach introduced in Eq. (4) is suitable for applications where the CFD volume nodes need to be deformed using the displacement of the CFD surface nodes. However, the surface mesh deformation method we introduce in this paper deforms the surface mesh nodes given the displacements on *curves*, not points. Therefore, this point-based inverse distance weighting approach is not suitable for the proposed method.

To demonstrate the shortcomings of this point-based deformation method in our application, we consider the following example. Suppose we have four nodes and three line elements that define a piecewise linear curve. We want to deform a point that is on the curve using the displacements defined on this curve. However, the point is positioned in the middle of one line element, as shown in Fig. 10a. In our surface mesh deformation process, we want this point to precisely follow the displacement of the curve so that it remains on the curve. As an example design change, we move the center two points that define the curve upward, while the two points on the two edges of the curve are moved downward. After this change, the updated coordinates of the points that define the feature curves are as shown in Fig. 10b. If we use a point-based deformation approach, the point initially on the curve does not follow the curve directly. This is because the displacement of this point is affected by the displacement of all points that define the curve.

This example is analogous to having CFD surface nodes on the feature curves of the original design. While we may have many CFD nodes on these intersection and feature curves, these CFD surface nodes are not necessarily co-located with the points that define these curves. As a result, if we use a point-based deformation approach, the CFD nodes initially on the intersection and feature curves are not guaranteed to follow these curves' displacements. As a result, the deformed surface mesh nodes do not accurately represent these curves, yielding an invalid surface mesh.

To address this problem, we propose a curve-based mesh deformation approach. With this approach, instead of a summation over the points, we use an integral over curves. This is equivalent to a summation over an infinite collection of points that form $\mathcal{C}_{S_0}$. This curve-based deformation method can be written as

$$\Delta \mathbf{x}_{\text{deform},i} = \frac{\int_{\mathcal{C}_{S_0}} w(\mathbf{x}_{S_0,i}, t) \Delta \mathcal{C}_{S_0}(t) L \mathrm{d}t}{\int_{\mathcal{C}_{S_0}} w(\mathbf{x}_{S_0,i}, t) L \mathrm{d}t}, \tag{6}$$

where we use the parametric variable $t \in [0, 1]$ to traverse these curves and L is the total length of the curve $\mathcal{C}_{S_0}$. Using this approach, the weight w now depends on both the coordinate of evaluation and the parameter t. This weight can be written as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

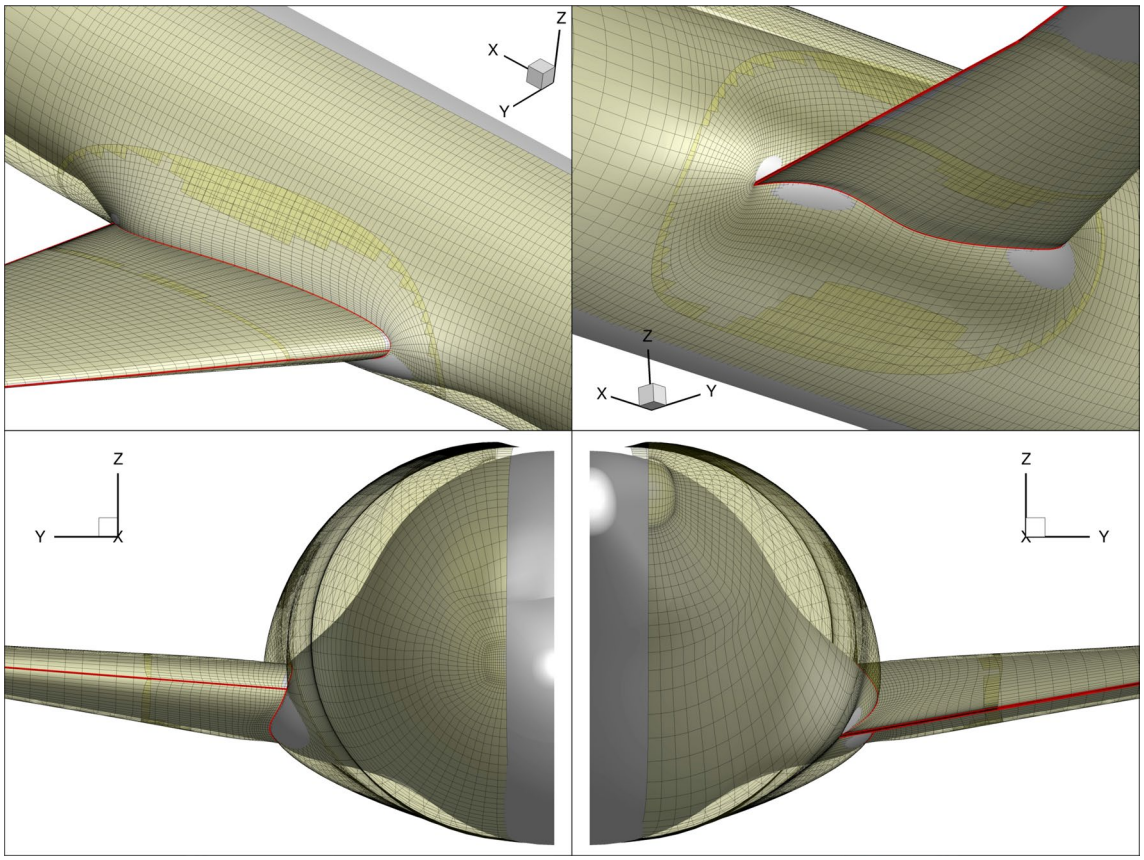


Fig. 11 Resulting surface mesh using the displacements of $\Delta\mathbf{x}_{\text{deform}}$. While the surface mesh conforms with the intersection and feature curves, it does not conform to the surfaces

$$w(\mathbf{x}_{S_0,i}, t) = \frac{1}{\left|\left|\mathbf{x}_{S_0,i} - \mathcal{C}_{S_0}(t)\right|\right|_2^m}, \tag{7}$$

where $\left|\left|\mathbf{x}_{S_0,i} - \mathcal{C}_0(t)\right|\right|_2$ is the distance between the point on the curve $\mathcal{C}_{S_0}$ defined at parameter t, and similarly to the point-based approach, we raise this expression to the power of m. Again, we set $m = 3$ for simplicity; however, combinations of other m values could be used. Using this curve-based deformation approach, we can see that the point that was initially on the curve in our example problem follows the curve after the curve is displaced, as shown in Fig. 10c. In Appendix A we detail the computation of the terms in Eq. (6).

Using Eq. (6), we compute $\Delta\mathbf{x}_{\text{deform}}$ for every point in the original surface mesh. With this approach, the intersection and feature curves are the *seeds*, and the surface mesh nodes are the *evaluation points* in the curve-based deformation method. The resulting surface mesh using $\Delta\mathbf{x}_{\text{deform}}$ to deform the surface mesh nodes for the example configuration is shown in Fig. 11. In the same figure, we also plot the updated intersection and feature curves, and the figure demonstrates how the nodes that were initially on these feature curves follow the curves, while the nodes away from these curves are deformed to maintain a consistent mesh topology. However, while the nodes near the intersection and feature curves conform to the updated geometry, the nodes in other locations on the geometry do not.

2.6 Combining the deformations

After obtaining the two displacement fields $\Delta\mathbf{x}_{\text{FFD}}$ and $\Delta\mathbf{x}_{\text{deform}}$, we combine these using a blending function. The combined displacement for the ith surface mesh node can be written as

$$\Delta\mathbf{x}_{\text{blend},i} = f(d_i)\Delta\mathbf{x}_{\text{FFD},i} + (1 - f(d_i))\Delta\mathbf{x}_{\text{deform},i}, \tag{8}$$

where $f(d_i) \in [0, 1]$ is the blending function, and d_i is the distance between the ith surface mesh node and the intersection curve of the baseline design.

This step has three goals: First, we want the resulting displacement field to be equal to the displacement field given by $\Delta\mathbf{x}_{\text{deform}}$ on the intersection curve. Second, we want the displacement field to be equal to the displacement field given by $\Delta\mathbf{x}_{\text{FFD}}$ away from the intersection curve. Third, we want to

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

have a smooth transition from $\Delta\mathbf{x}_{\text{deform}}$ to $\Delta\mathbf{x}_{\text{FFD}}$ as the distance from the intersection curve increases. To achieve these goals, we define the blending function to have the value of one for large d_i values and a value of zero where $d_i = 0$. While there are many ways to define this function with these specifications, we define f over $d_i \in [0, \infty)$ as

$$f(d_i) = \begin{cases} 0.5\left(\frac{d_i}{d^\star_{\text{comp}}/2}\right)^3 & \text{if} \quad d_i < \frac{d^\star_{\text{comp}}}{2}, \\ 1 - 0.5\left(\frac{d^\star_{\text{comp}} - d_i}{d^\star_{\text{comp}}/2}\right)^3 & \text{if} \quad \frac{d^\star_{\text{comp}}}{2} \le d_i < d^\star_{\text{comp}}, \\ 1 & \text{otherwise.} \end{cases} \tag{9}$$

This formulation satisfies the requirements for f, while being C_1 continuous over the whole domain. In this approach, we define a $d^\star_{\text{comp}}$ value for each component, which is used to control the distance where the contribution from $\Delta\mathbf{x}_{\text{deform}}$ is propagated on the surface mesh. When computing $f(d_i)$ for the ith surface mesh node, we use the $d^\star_{\text{comp}}$ value of the component that the point was mapped to, as described in Sect. 2.2.

For CFD surface nodes on either component, if the distance to the intersection curve is greater than $d^\star_{\text{comp}}$, then f has the constant value of one. Therefore, there is no contribution from $\Delta\mathbf{x}_{\text{deform}}$ when computing $\Delta\mathbf{x}_{\text{blend}}$. Because of this, there is no need to compute $\Delta\mathbf{x}_{\text{deform}}$ for most of the surface nodes, and we only compute this term if the distance to the intersection curve is smaller than the $d^\star_{\text{comp}}$ of the component where this point was mapped. This approach reduces the number of unnecessary computations because computing $\Delta\mathbf{x}_{\text{deform}}$ is one of the most costly parts of the method, as shown in Sect. 3.2.

The $d^\star_{\text{comp}}$ controls how far from the intersection curve the effects of the curve-based deformation are propagated. Therefore, the selection of this parameter for each component is essential. On the one hand, a larger $d^\star_{\text{comp}}$ value increases the range of design changes that can be achieved with the method, as presented in Sect. 3.1. On the other hand, a larger $d^\star_{\text{comp}}$ value also increases the area where the effects of the surface mesh deformation algorithm is propagated. This can result in the undesired displacement of some surface mesh nodes, and additional feature curves need to be included in the curve-based displacement step to avoid this problem.

For example, the surface mesh nodes on the top and bottom of the fuselage geometry lie on the symmetry plane used for the CFD simulations, as shown in Fig. 1b. If the $d^\star_{\text{fuselage}}$ parameter is selected such that the surface nodes on the symmetry plane is within this distance from the intersection curve, $\Delta\mathbf{x}_{\text{blend}}$ for these nodes include a contribution from $\Delta\mathbf{x}_{\text{deform}}$. This moves these nodes away from the symmetry plane as the design is changed; however, these nodes must stay on the symmetry plane to obtain a valid CFD mesh. There are two approaches to avoid this problem: First, the curves that define the upper and lower edge of the fuselage can be included in the set of feature curves introduced in Sect. 2.4. This would preserve the location of the points that were originally on the symmetry plane; however, this approach increases the complexity of setting up the method for a configuration and increases the computational cost of updating the surface mesh. The second approach is to pick a $d^\star_{\text{fuselage}}$ value that is smaller than the distance from the intersection curve to any of the surface nodes that are on the symmetry plane. This approach limits the range of design change that can be achieved; however, it also limits the area where the surface mesh deformation algorithm's effects are propagated. As a result, the $\Delta\mathbf{x}_{\text{blend}}$ for the nodes on the symmetry plane does not include a contribution from $\Delta\mathbf{x}_{\text{deform}}$, and the nodes remain on the symmetry plane after the design is changed.

With the fuselage component, we use the second approach and pick a $d^\star_{\text{fuselage}}$ that is smaller than the minimum distance from the intersection curve to the nodes on the symmetry plane. With the example configuration we use herein, we use a $d^\star_{\text{fuselage}}$ value that is equal to 80.86% of the fuselage radius. Similarly, we set the $d^\star_{\text{wing}}$ value to 25.61% of the semi-span, which is smaller than the distance from the intersection curve to the Yehudi break. This limits the changes on the surface mesh nodes on the wing to remain within the inboard portion of the wing before the Yehudi break. This value is larger than the distance from the intersection curve to the wing nodes on the symmetry plane, and we include the edges of the wing on the symmetry plane in the set of feature curves as described in Sect. 2.4 to maintain the nodes on the symmetry plane.

The combined displacement field $\Delta\mathbf{x}_{\text{blend}}$ for the example wing-body configuration is shown in Fig. 12. The surface mesh conforms to the intersection curves; however, it does not conform to the component geometries near the intersection. Away from the intersection curve, where $f(d_i)$ is equal to one, the cells conform to the component geometries since the displacement does not include any contribution from the curve-based deformation. As a result of this step, the surface mesh nodes on the intersection curve and the nodes away from the intersection curve conform to the updated geometry, while the points that are near the intersection curve but not on it do not conform to the updated geometry. However, despite this discrepancy, this intermediate stage mesh is used for further operations introduced in the following sections to obtain a surface mesh that conforms with the updated geometry everywhere.

While the deformation $\Delta\mathbf{x}_{\text{blend}}$ conforms with the updated intersection curve, and the component geometries away from the intersections, it does not necessarily

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

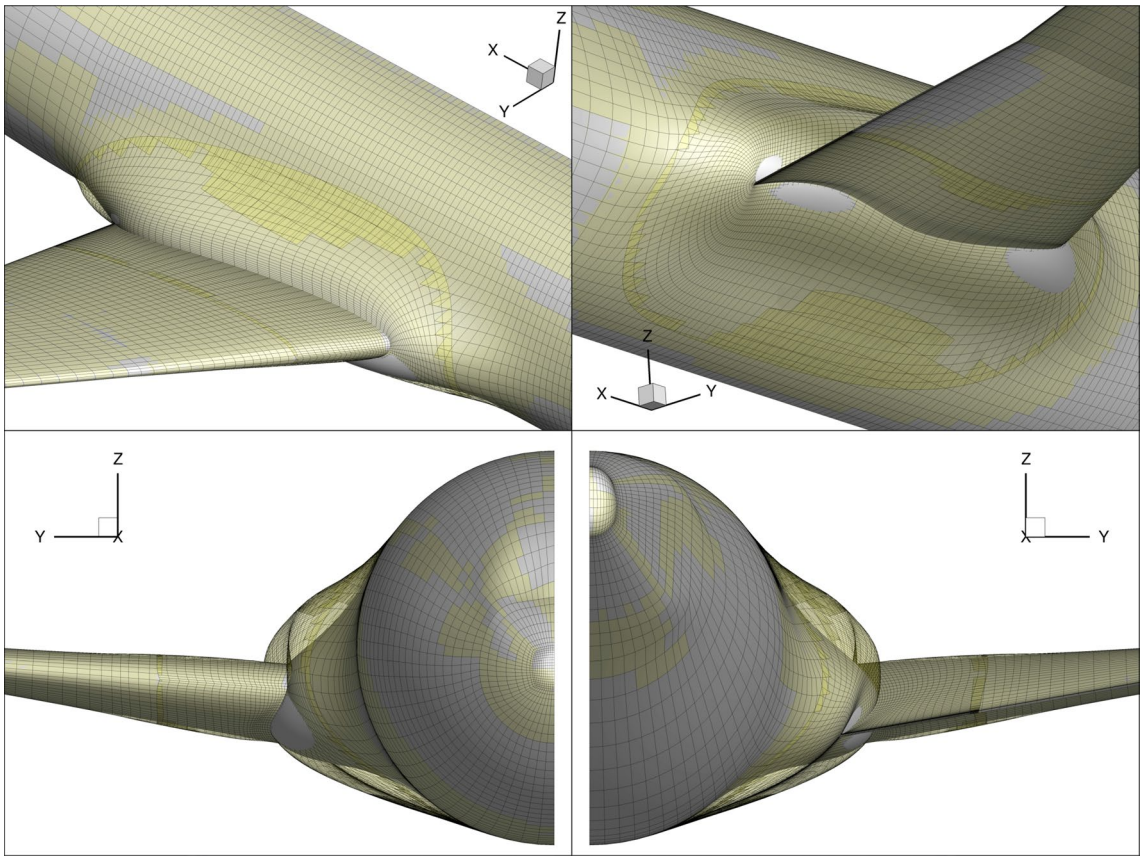


Fig. 12 Resulting displacement field after blending between the FFD volume and curve-based mesh deformation displacements. The displaced surface mesh is shown translucent yellow, while the component geometries are shown in gray. With these displacements, the mesh conforms with the intersection curve itself, while the surface mesh nodes do not conform to the component geometries near the intersection curve. Similarly, the surface mesh nodes conform to the component geometries away from the intersection (color figure online)

conform to the updated feature curves. The example design change we introduced in Sect. 2.2 is not sufficient to demonstrate this effect because all the feature curves are straight lines. Therefore, to demonstrate this type of discrepancy, we introduce an arbitrary modification to the twist distribution on the wing, in addition to the fuselage perturbation, as shown in Fig. 13a. With this design change, we twist the wing in the positive direction in the region between the fuselage and the Yehudi break, while twisting the wing in the negative direction near the wing root and tip. In this context, a positive twist moves the wing leading edge up and the trailing edge down with respect to the rotation axis, which is defined as the quarter-chord line.

The upper and lower trailing edge curves with the original design are straight lines with no curvature. By modifying the wing twist distribution, we create a curvature on the upper and lower trailing edge curves to highlight the discrepancy between the geometry and $\Delta\mathbf{x}_{\text{blend}}$. When we plot $\Delta\mathbf{x}_{\text{blend}}$ using this design change, we can see that the surface mesh nodes that were initially on the upper and lower trailing edge curves do not conform to the updated feature curves (Fig. 13b). This discrepancy is because of the curvature on the trailing edge, along with the span-wise perturbation of the intersection curve as we modify the fuselage design.

2.7 Projection to feature curves

To remove the discrepancy shown in Fig. 13b, we project the points that were initially located on the intersection and feature curves onto the updated locations of the curves after displacing these nodes using $\Delta\mathbf{x}_{\text{blend}}$. To do this, we need to determine which surface points are located on the intersection and feature curves during the setup phase of the method. We do this by determining the points that are within a small tolerance from the original curves $\mathcal{C}_{S_0}$. We denote the original locations of these points as $\mathbf{x}_{\mathcal{C}_{S_0}}$. The updated coordinates of these points after the projection can be written as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

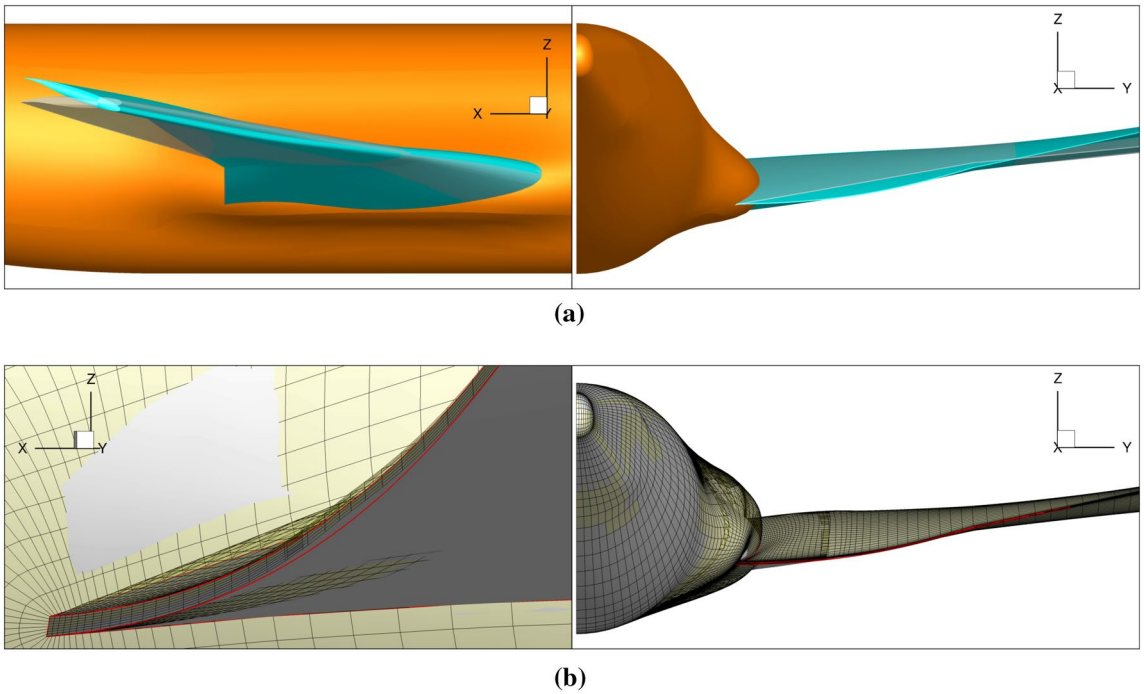


Fig. 13 The change in wing twist creates a curvature in the trailing edge curves, as seen in **a**. We use this design change to demonstrate how displacing the surface mesh nodes using $\Delta \mathbf{x}_{\text{blend}}$ does not guarantee that points that were initially on the feature curves to remain on the corresponding curves. **a** Geometry with the modified twist-distribution for the wing, along with the perturbation to the fuselage introduced in Fig. 3b. The updated wing geometry is shown in blue, while the original wing geometry is shown in translucent gray. The fuselage geometry is shown in orange. The modified twist distribution creates a curvature to the originally straight trailing edge. **b** Surface mesh displaced with $\Delta \mathbf{x}_{\text{blend}}$, along with the true geometry of the component. The surface mesh is shown in translucent yellow, while the true geometry is shown in blue. The intersection and feature curves of the true geometry are shown in red. The figure demonstrates how the CFD surface mesh nodes that were initially on these feature curves are not located on the curves after displacing them with $\Delta \mathbf{x}_{\text{blend}}$ (color figure online)

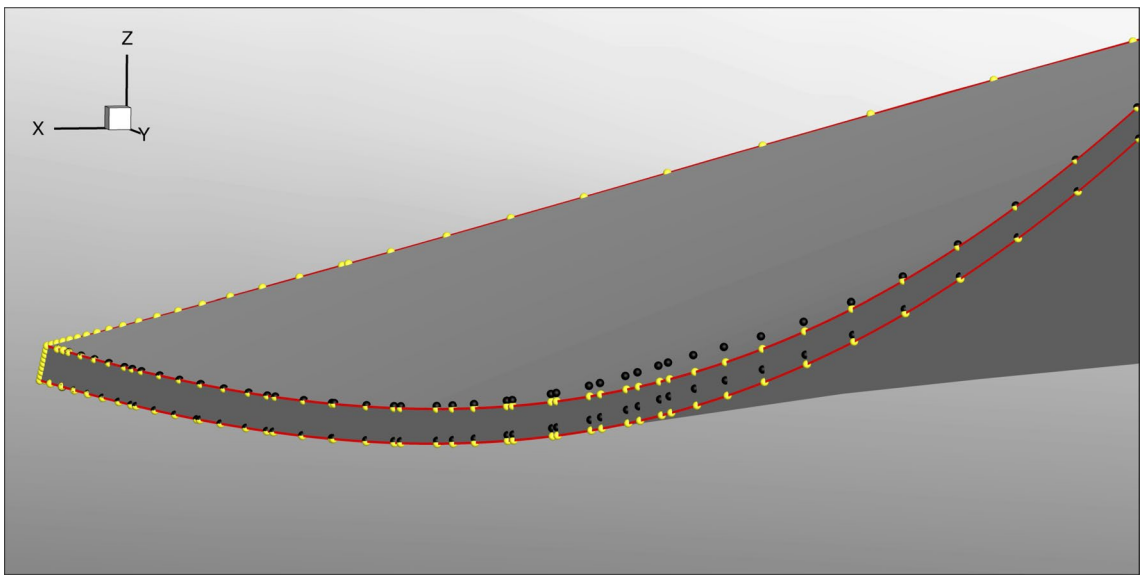


Fig. 14 The surface mesh nodes that were located on the feature curves with the original design. The location of these points after displacing them with $\Delta \mathbf{x}_{\text{blend}}$ is shown in black, while the projected points are shown in yellow. Red lines show the updated intersection and feature curves (color figure online)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

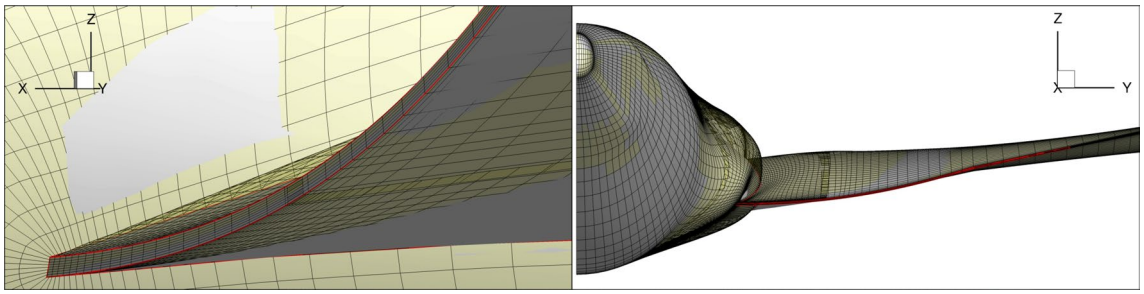


Fig. 15 Surface mesh displacements resulting from the point-based deformation approach used to propagate the changes introduced with the curve projections. The surface mesh now conforms with the updated intersection and feature curves; however, it still does not conform to the updated component surfaces

$$\mathbf{x}_{\mathcal{C}} = \mathcal{P}_{\mathcal{C}}\left(\mathbf{x}_{\mathcal{C}_{S_0}} + \Delta\mathbf{x}_{\text{deform},\mathcal{C}_{S_0}}\right), \tag{10}$$

where $\Delta\mathbf{x}_{\text{blend},\mathcal{C}_{S_0}}$ are the blended displacements for the points originally on $\mathcal{C}_{S_0}$, and $\mathcal{P}_{\mathcal{C}}$ is the operator for the projection onto the updated curves $\mathcal{C}$.

The projection resulting from the design change shown in Fig. 13a is shown in Fig. 14. Because the CFD surface nodes on the feature curve are located on the intersection curve even after displacing them with $\Delta\mathbf{x}_{\text{blend}}$, we do not need to project these back to the intersection curve. Each point is projected to the intersection curve it was initially on, and this prevents points that were initially on the lower trailing edge from getting projected to the upper trailing edge curve.

2.8 Point-based deformation

The displacement of CFD nodes resulting from the projection step introduced in the previous section can be written as

$$\Delta\mathbf{x}_{\mathcal{C}} = \mathbf{x}_{\mathcal{C}} - \left(\mathbf{x}_{\mathcal{C}_{S_0}} + \Delta\mathbf{x}_{\text{blend},\mathcal{C}_{S_0}}\right). \tag{11}$$

This displacement is only defined for the points that were originally on intersection and feature curves. Therefore, we need to propagate these changes to the points that are not originally located on these curves. We denote the coordinates of CFD surface nodes that are not mapped to the intersection or feature curves with the initial design as $\mathbf{x}_{\mathcal{C}'_{S_0}}$ and the union of $\mathbf{x}_{\mathcal{C}_{S_0}}$ and $\mathbf{x}_{\mathcal{C}'_{S_0}}$ is the full set of CFD surface points $\mathbf{x}_{S_0}$.

Using the displacements of the points that were projected to updated feature curves, we deform the remaining CFD nodes using a point-based deformation approach. In this step, the projected points are the *seeds*, and the remaining surface mesh nodes are the *evaluation points*. The displacement for the ith node of the points that were not projected to these curves can be written as

$$\Delta\mathbf{x}_{\mathcal{C}',i} = \frac{\sum_j w_j(\mathbf{x}_{\mathcal{C}'_{S_0},i})\Delta\mathbf{x}_{\mathcal{C},j}}{\sum_j w_j(\mathbf{x}_{\mathcal{C}'_{S_0},i})}, \tag{12}$$

which is similar to the formulation introduced in Eq. (4). In this formulation, we compute the weights w_j as

$$w_j(\mathbf{x}_{\mathcal{C}'_{S_0},i}) = \frac{1}{\left|\left|\mathbf{x}_{\mathcal{C}'_{S_0},i} - \mathbf{x}_{\mathcal{C}_{S_0},j}\right|\right|_2^m}, \tag{13}$$

where, $||\mathbf{x}_{\mathcal{C}'_{S_0},i} - \mathbf{x}_{\mathcal{C}_{S_0},j}||_2$ is the Euclidean distance between the evaluation coordinate $\mathbf{x}_{\mathcal{C}'_{S_0},i}$, and $\mathbf{x}_{\mathcal{C}_{S_0},j}$ is the jth point that was projected to the intersection and feature curves in the previous step. We only use this point-based deformation for points on components with a feature curve included in the surface mesh deformation approach. If the ith point is on a component that does not have a feature curve that is near the intersection, then we set $\Delta\mathbf{x}_{\mathcal{C}',i}$ to zero. Once these displacements are included, we refer to the CFD surface mesh node coordinates as $\mathbf{x}_{\text{curve}}$. This is because the surface mesh conforms with the updated intersection and feature curves, and it does not necessarily conform to the updated surfaces.

Figure 15 demonstrates the resulting surface mesh after this point-based deformation step. We have moved the CFD surface mesh nodes to conform to both the intersection and feature curves due to the curve-projection and point-based deformation. In the example configuration, because we do not have any feature curves on the fuselage, we do not compute the point-based deformation displacement introduced in Eq. (12) for the nodes that were initially on the fuselage geometry. At this stage, while the surface mesh nodes that were initially on the intersection and feature curves conform to the updated curves, the surface mesh itself does not conform to the component surfaces, as shown in Fig. 15.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

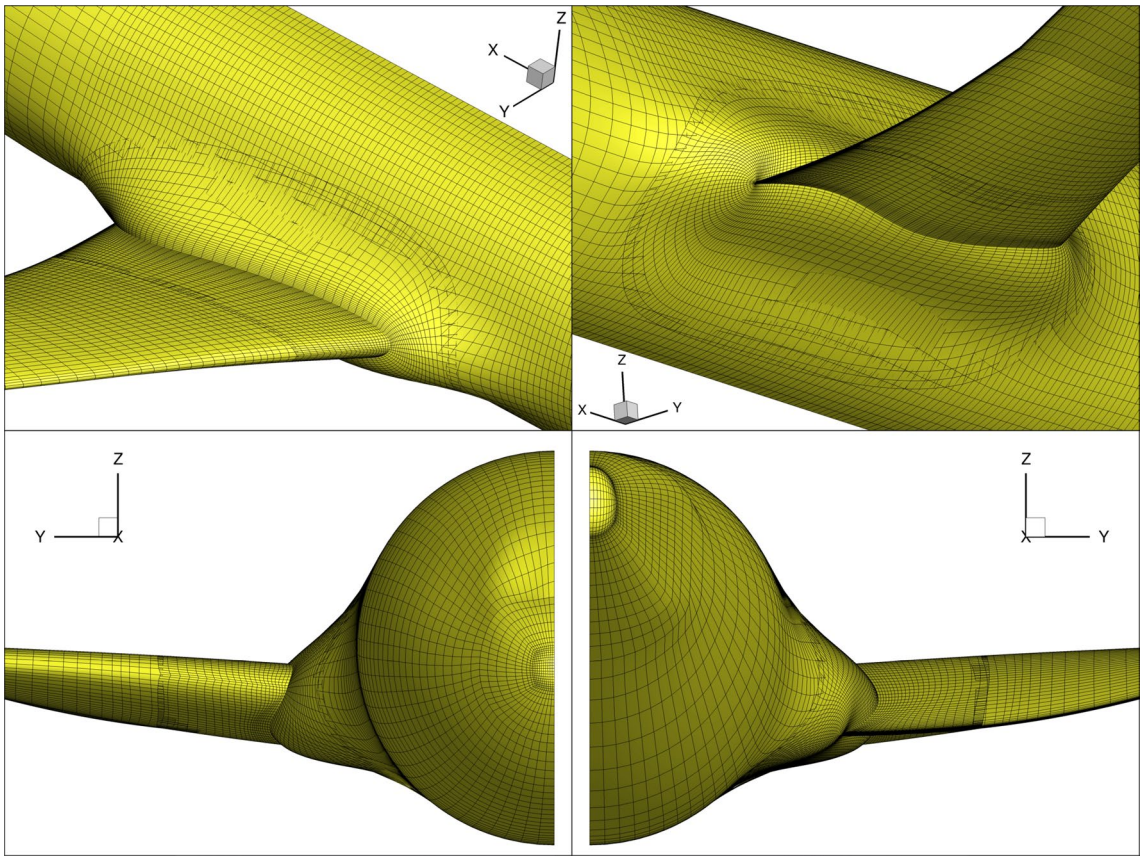


Fig. 16 Updated CFD surface mesh after projecting the nodes to the component surfaces using the design change introduced in Fig. 13a. The updated surface mesh conforms both with the updated curves and surfaces

2.9 Projection back to surfaces

As the final step of the method, we project the CFD surface nodes $\mathbf{x}_{\text{curve}}$ to the updated component surfaces S. This process can be written as

$$\mathbf{x} = \mathcal{P}_S(\mathbf{x}_{\text{curve}}), \tag{14}$$

where $\mathcal{P}_S$ is the projection operator to the updated component surfaces. Because $\mathbf{x}_{\text{curve}}$ conforms with the intersection and feature curves, the projection of the nodes located on these curves are simply the previous coordinates of these points, while the points that were originally located on the surfaces are projected to obtain an accurate representation of the underlying geometry. We obtain a valid CFD surface mesh that conforms with both the updated intersection and feature curves and the updated component surfaces after this operation.

Figure 16 shows the final CFD surface mesh that would be obtained from the design change introduced in Fig. 13a. Starting from the initial CFD surface mesh shown in Fig. 1b, the method *deforms* the CFD surface nodes that are within $d^{\star}_{\text{comp}}$ of the intersection curve on both components to conform to the updated geometry. The nodes that are located more than $d^{\star}_{\text{comp}}$ from the intersection are only moved with the deformation of the FFD volumes. After this step, we can use a volume mesh deformation approach [44] with the final displacements of the surface mesh to obtain an updated volume mesh for the CFD solver.

2.10 Summary and generalization of the proposed method

The proposed surface mesh deformation method consists on the following steps to deform an original CFD surface mesh to conform to the updated geometry of the design:

1. Compute geometric design changes for each component using a component-based tool (Sect. 2.2).
2. Update the intersection curve between the intersecting components (Sect. 2.3).
3. Update the feature curves that are included in the curve-based deformation performed in the next step (Sect. 2.4).

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

4. Compute the displacements of the surface mesh nodes with respect to the displacements of the intersection and feature curves using the curve-based mesh deformation approach (Sect. 2.5).
5. Combine the deformations from the design changes for each component and the curve-based deformation using a blending function (Sect. 2.6).
6. Project the points that were originally on intersection and feature curves onto the updated curves (Sect. 2.7).
7. Deform the points that were originally on surfaces with a point-based deformation approach using the displacements of the points in the previous step (Sect. 2.8).
8. Project the resulting points onto the updated component surfaces (Sect. 2.9).

In this work, we demonstrate each step of the method with our implementation using the FFD approach [15] for the component-based design changes, and the pySurf module [12] to perform the geometric operations required for the method. However, for each step of the method, the implementation can use other tools and approaches. For Step 1, we can use a CAD or CDG tool instead of the FFD approach. With these tools, the triangulated surface mesh can still be tracked, or the continuous geometry representations available can be used to perform the geometric operations needed by the method. The mesh deformation approaches used in Steps 4 and 7 can be extended to include additional terms that account for the local design changes, element rotations, or both [44]. Finally, the blending function used in Step 5 can be modified to obtain a better displacement field before the projection steps.

The method is applicable to arbitrary design changes and geometries as long as the geometric topology does not change. The example configuration we introduced in Sect. 2.1 is representative of a wide range of design problems that arise with the design of commercial aircraft configurations. While this configuration only has a single intersection curve, this method can be generalized for configurations with multiple intersections.

There are two possible approaches to handle geometries with multiple intersections. If the multiple intersections are sufficiently far from each other so that the localized surface mesh deformation for one intersection does not affect the other, the two intersections can be handled independently. Our implementation of the method uses an object-oriented approach, and therefore, we can include an arbitrary number of intersection regions for each configuration. On the other hand, if two or more intersections are close to each other, so that the localized changes affect overlapping regions, the approach needs to be modified. For such cases, we can consider the localized changes introduced by the combination of multiple intersections, rather than treating each intersection separately. To accomplish this, the multiple intersections and feature curves can be considered simultaneously in the curve-based deformation performed in Step 4. However, the distance from the intersection curve required for the blending function used in Step 5 should be modified to be the distance from the *closest* intersection curve. With these modifications, the method can be used to handle configurations where there are multiple intersections.

One of the main challenges of implementing the proposed method is the computation of the updated intersection and feature curves. At each design point, these curves need to be recomputed using a consistent topology to compute the displacements of each line element that defines the original curve. For the example configuration, we introduced in Sect. 2.1, this can be done by re-meshing the updated intersection curve between the features on the wing geometry. For wings with blunt trailing edges, these are the leading edge, along with the upper and lower trailing edges. For arbitrary configurations, a consistent mapping of the intersection curves between design iterations can either use features on components as we introduced or use features of the underlying CAD geometry to compute the displacement of the intersection and feature curves.

3 Results

This section demonstrates the range of design changes that can be achieved with the method and analyzes its computational cost.

3.1 Range of design changes

The proposed surface mesh deformation method can handle arbitrary design changes for either of the intersecting components. Furthermore, the method provides an approach to model the parts of the components that are originally located inside another component, unlike some previous methods where a reconstruction or an extrapolation step is required to represent parts of the geometry for such cases [33]. This is achieved with the triangulated surface mesh of each component. For example, the wing-root geometry of the DLR-F6 configuration is originally located inside the fuselage. However, suppose any design change exposes this geometry by moving these surfaces outside the fuselage component. In that case, the method is still able to accurately represent the region of the wing geometry that was originally inside the fuselage.

Despite the flexibility of handling arbitrary design changes, the surface mesh deformation method introduced herein has two main factors that limit the range of geometric design changes that can be achieved. First of all, while the method guarantees that the updated surface mesh conforms with the updated geometry, the surface mesh cells' area and

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

quality are not guaranteed to be preserved. As the geometry is modified from the initial geometry, the overall quality of the mesh degrades because the initial surface mesh was tailored for the initial geometry. This imposes a limit on the range of the geometric changes that yield a valid surface mesh.

Secondly, the method cannot handle changes that modify the topology of the geometry. As previously mentioned in Sect. 1.2, a consistent geometric topology means that the components that intersect each other with the original design also intersect when the design is updated, and the components that do not intersect originally do not form a new intersection as the design is updated. While the previous limitation due to the degradation of the surface mesh quality is the most common bottleneck that limits the range of design changes, any design change introduced with the method must also preserve the configuration's geometric topology.

This section quantifies the range of design changes that can be achieved with the proposed method using two example configurations. We use a structured overset surface mesh for both of these configurations and compute two metrics for every cell in the surface mesh. The first metric of interest is the area of the cells. To compute each cell area, we first compute the cross product between the two diagonal vectors on each element, and take half of the magnitude of the resulting vector. The second metric of interest is the cell quality, and to compute the quality of the quadrilateral surface cells, we use an adaptation of the $2 \times 2 \times 2$ determinant metric used in the ICEM CFD software [51]. To compute this metric, we compute the area of the element at all four corners by computing the magnitude of the cross product using the two edges that define the corner. Using these four area values for each cell, we compute the quality as the ratio of the minimum area divided by the maximum area computed on the four corners of the element.

For each example configuration, we compute the area and quality for every surface cell using the baseline design, written as $A_{0,i}$ and $Q_{0,i}$ for the ith cell, respectively. After modifying the surface mesh using the proposed method, we again compute the area and quality, written as A_i and Q_i for the ith cell, respectively. For a valid updated CFD surface mesh, we enforce the requirement that the cell area and quality for any cell do not go below 10% of the original value for that cell. To enforce this, we define the minimum ratios of area $R_{\min,A}$ and quality $R_{\min,Q}$ as

$$\begin{aligned} R_{\min,A} &= \min_i \frac{A_i}{A_{0,i}}, \\ R_{\min,Q} &= \min_i \frac{Q_i}{Q_{0,i}}, \end{aligned} \tag{15}$$

where both ratios are equal to one when there are no geometric design changes. For the example cases, we demonstrate the limit for any design change is determined by finding the point where either $R_{\min,A}$ or $R_{\min,Q}$ first reach a value of 0.1.

3.1.1 Wing-body configuration of a commercial transport aircraft

The first example configuration is the DLR-F6 wing-body geometry introduced in Sect. 2.1. Despite being a relatively simple configuration, the intersection between the wing and the fuselage represents a wide range of geometries that exhibit intersections between components. In this work, we use the geometry introduced by Brodersen [46], a wind-tunnel model of a twin-engine wide-body aircraft. The fuselage diameter and wing-span of the model are 148.4 mm and 1171.3 mm, respectively. The wing trailing edge has a kink located at 235.1 mm span-wise from the fuselage centerline, and the root chord is 240.1 mm.

With this configuration, we consider four geometric design changes: the fuselage perturbation introduced in Fig. 3a, wing mounting angle about the quarter chord, and horizontal and vertical displacement of the wing with respect to the fuselage. Fig. 17 shows the changes in the minimum ratio of area and quality as the design is modified. For the design changes introduced here, the cells' minimum area ratio decreases faster than the minimum quality ratio. For larger design changes, this can lead to volume mesh cells with negative volumes in the final CFD mesh. Despite the reduction in cell areas, the method can obtain a valid surface mesh as the geometry is modified significantly.

Figure 18 shows the limiting design change for the fuselage perturbation introduced in Fig. 3a. This limit of 50.50 mm is larger than the original perturbation of 40 mm used in Sect. 2. In this figure, the contours represent the change in area for each cell compared to the original surface mesh. The color red represents the regions where the surface cell areas increase. The color blue represents the regions where the surface cell areas decrease compared to the original surface mesh. This figure shows two bands around the wing mesh with multiple cells that obtain a smaller area. The outboard band is due to the blending function, while the inboard band is due to the nonlinear geometric change. Furthermore, this design change increases the cell areas on the upper and lower side of the fuselage near the wing. This is an extreme example that visualizes the limits of the design change; however, the original CFD surface mesh can be modified to increase resolution in these areas to prevent loss of accuracy in the CFD simulations using such design changes.

Similarly, Fig. 19a–c show the range of design changes for the wing mounting angle, horizontal and vertical displacement of the wing, respectively. In these figures, we

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

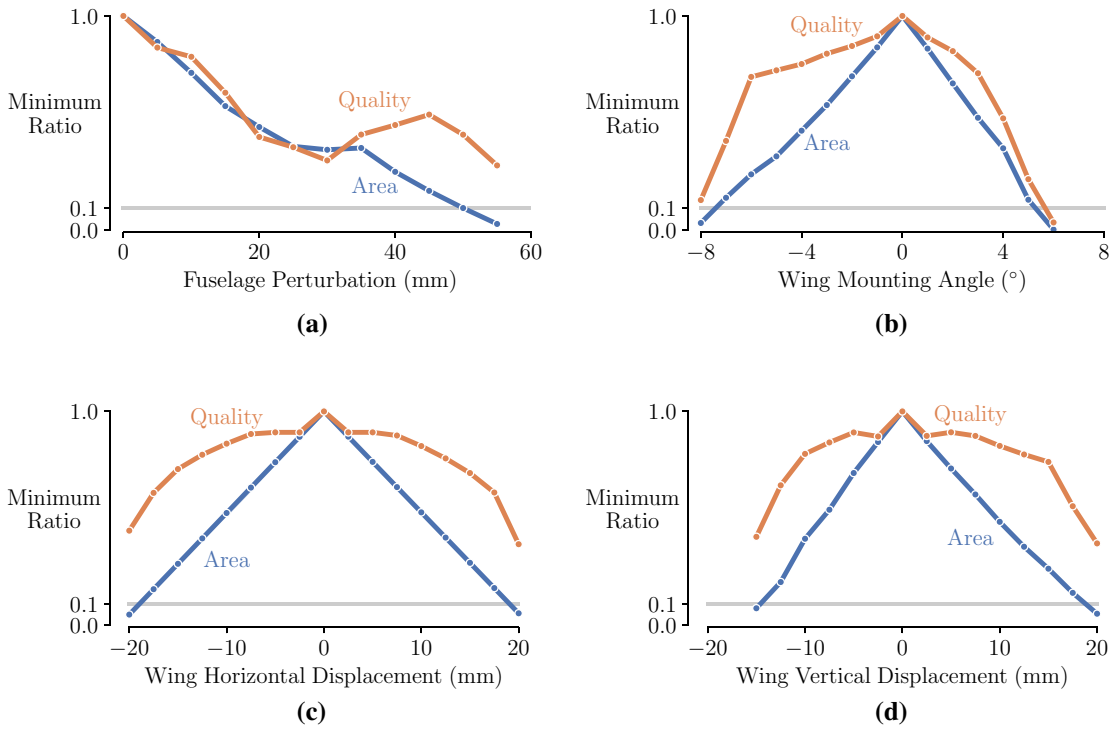


Fig. 17 Minimum ratio of area and quality over each cell as the design is changed. The minimum ratio metric corresponds to the largest reduction in the area or quality metrics for any cell in the surface mesh compared to the baseline design. In this work, we consider a mesh to be a valid surface mesh if both of the minimum ratio for area and quality are above 0.1. The model has a semi span of 585.65 mm and a fuselage diameter of 148.4 mm. **a** Fuselage perturbation introduced in Fig. 3a. The perturbation distance corresponds to displacing the FFD control points by that amount radially outward from the fuselage center. The displacement shown in Fig. 3b is 40 mm. **b** Wing mounting angle defined by the quarter chord line of the wing. To achieve this design change, every wing FFD control point (Fig. 2a) is rotated about the quarter-chord line. A positive mounting angle corresponds to displacing the leading edge upward and the trailing edge downward. **c** Horizontal displacement of the wing. A positive displacement corresponds to moving the wing toward the aft of the aircraft. **d** Vertical displacement of the wing. A positive displacement corresponds to moving the wing upward

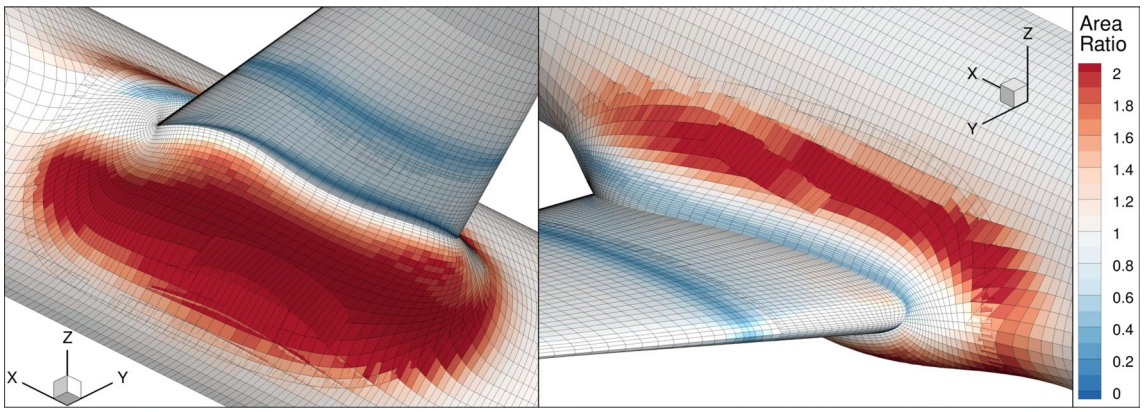
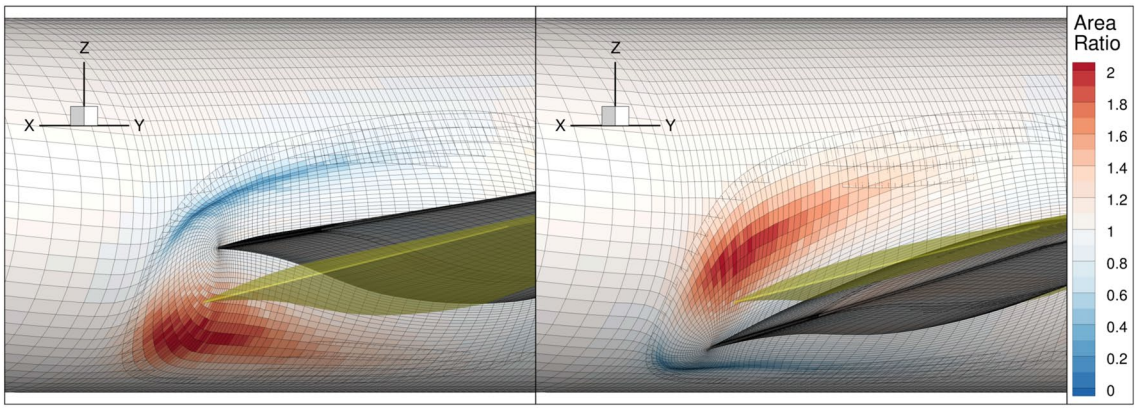
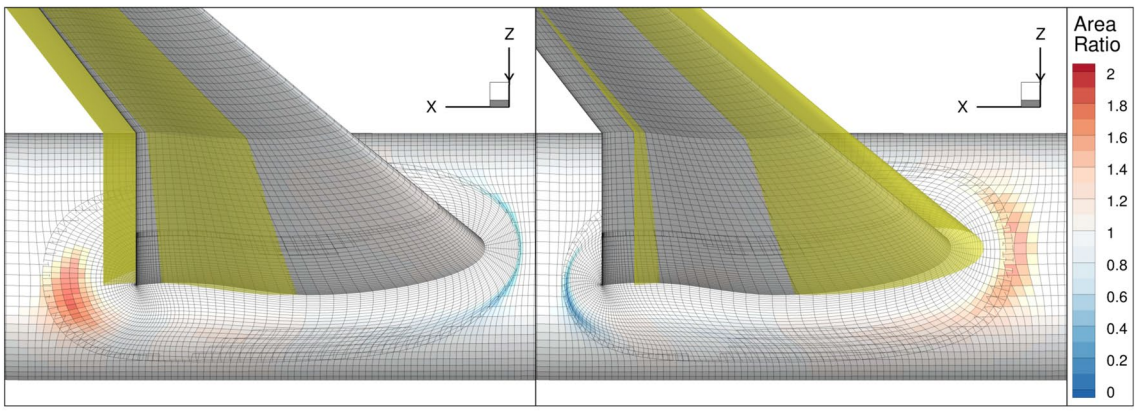


Fig. 18 Limit of fuselage perturbation, which is equal to perturbing all 16 FFD control points in Fig. 3a away from the fuselage by 50.50 mm, which is equal to 0.34 fuselage diameters. The contours visualize the change in area for each cell compared to the baseline surface mesh. Red regions highlight the cells that have a larger area, and blue regions highlight the cells that have a smaller area after the surface mesh update (color figure online)

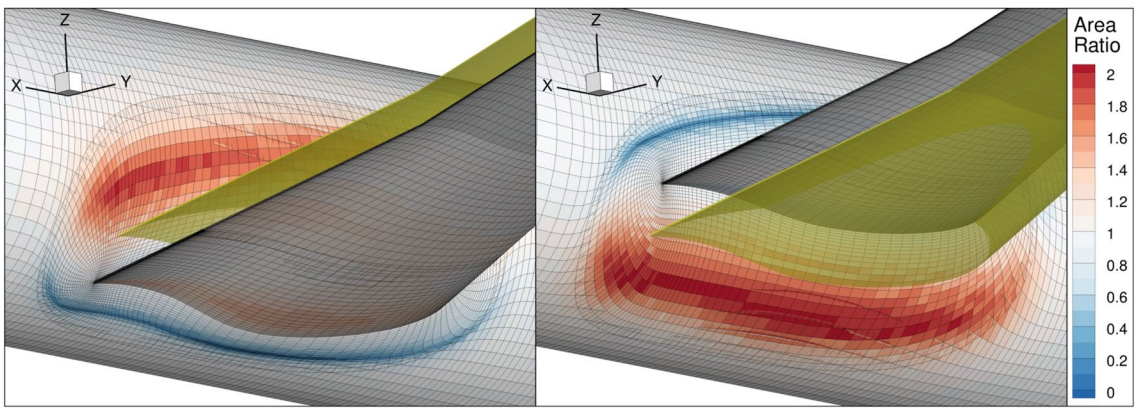
Content courtesy of Springer Nature, terms of use apply. Rights reserved.



(a)



(b)



(c)

Fig. 19 Range of design changes with the wing mounting position. The original wing position is shown in translucent yellow. In these figures, the contours visualize the change in area for each cell, compared to the baseline surface mesh. Red regions highlight the cells with a larger area, and blue regions highlight the cells with a smaller area after the surface mesh update. **a** Limits of wing mounting angle. The minimum (left) and maximum (right) limiting values are equal to − 7.42 and 5.46 degrees, respectively. Because the wing mounting angle is modified about the quarter-chord line, the trailing edge of the wing is displaced more than the leading edge, and this causes the fuselage cells near the trailing edge to be the limiting factor. **b** Limits of horizontal wing displacement. The minimum (left) and maximum (right) limiting values are equal to − 18.94 mm and 19.06 mm, respectively. These ranges are equal to − 7.89% and 7.94% wing-root chords, respectively. **c** Limits of vertical wing displacement. The minimum (left) and maximum (right) limiting values are equal to − 14.66 mm and 19.10 mm, respectively. These ranges are equal to − 6.10% and 7.95% wing-root chords, respectively (color figure online)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

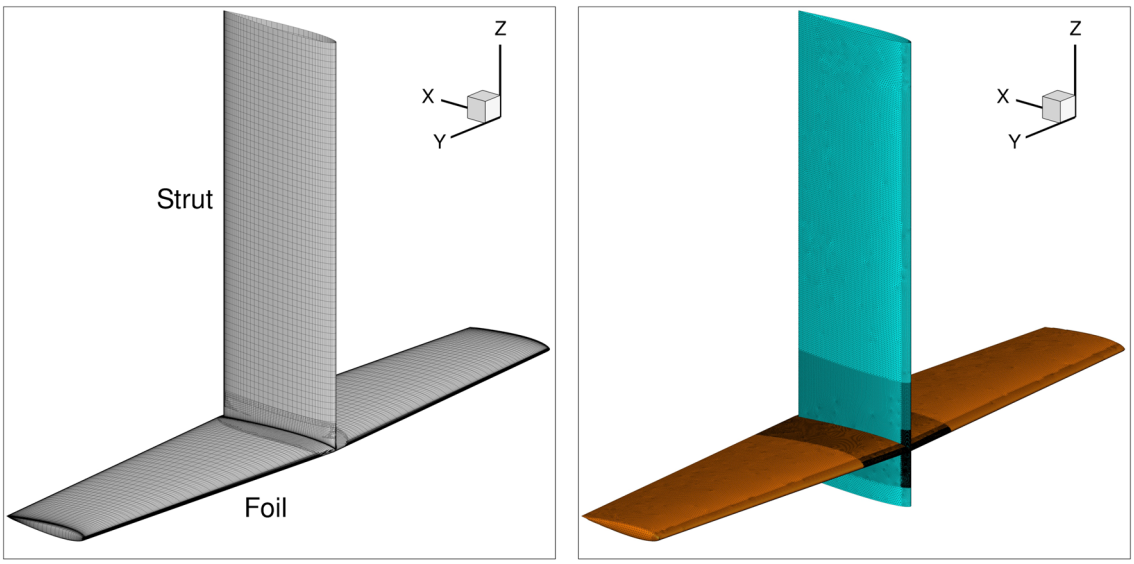
highlight the original location of the wing in translucent yellow and use the same color scheme to highlight the change in cell areas compared to the original surface mesh. For these design changes, the limiting factor is the region on the fuselage mesh, where the surface cells are decreasing due to the wing displacement. While the design ranges obtained with the current case is enough for high-fidelity design optimization, the ranges can be extended by increasing the $d^{\star}_{\text{fuselage}}$ parameter, which controls how far from the intersection curve the displacements are propagated on the fuselage surface mesh. In the current case, the $d^{\star}_{\text{fuselage}}$ is selected such that the cells on the symmetry plane near the lower fuselage are not affected by the curve-based deformation. To further increase the $d^{\star}_{\text{fuselage}}$ parameter for this case, we can include the feature curves that define this symmetry plane, and this would result in a wider range of possible design changes while preserving the symmetry plane used in the CFD simulations.

3.1.2 T-shaped hydrofoil

The second example configuration is a T-shaped hydrofoil (T-foil) geometry developed by Liao et al. [52], similar to the model presented by Binns et al. [53]. These hydrofoils are commonly used in marine applications [54–56]. We use this configuration to demonstrate the method's flexibility in applications other than an aircraft wing-fuselage intersection. The T-foil is composed of two distinct surfaces, as shown in Fig. 20a. The vertical surface is called the strut, and the horizontal one is called the foil. Both of these surfaces have a NACA 0012 cross-section. The foil geometry has a root chord length of 140 mm, a tip chord length of 95 mm, and a span of 660 mm. The strut geometry has a constant chord length of 135 mm and a height of 400 mm. Figure 20a also shows the overset surface mesh used for the T-foil, which contains 55,016 surface cells. The $d^{\star}$ parameters for the strut and the foil are set to 100 mm and 40 mm, respectively. Figure 20b shows the triangulated surface meshes used for the foil and strut, which contain 110,415 and 88,922 nodes and 220,826 and 177,550 elements respectively. Figure 21 shows the original foil-strut intersection curve, along with the feature curves we use for the surface mesh deformation method, which are the leading and trailing edge curves for the foil and the strut.

The design change of interest is foil rotation while preserving the strut geometry. We rotate the foil about the leading edge because of the tight spacing between the foil and the strut geometries near the leading edge, while the trailing edge of the intersection has some spacing on the foil geometry to allow for this design change, as shown in Fig. 21. The foil can also be translated in the vertical direction as it is rotated to achieve a similar effect of rotating about quarter chord. However, we only consider rotating the foil geometry in this example. Similar to the previous configuration, a positive rotation corresponds to moving the foil trailing edge downward.

The triangulated surface mesh for the strut is extended past the foil geometry to rotate the foil in the positive



(a) (b)

Fig. 20 CFD surface mesh and the triangulated surface meshes used for the T-foil configuration. **a** Surface mesh for the T-foil configuration. Only the active cells in the overset mesh are shown. The geometry consists of two sections: the strut (vertical surface) and the foil (horizontal surface). **b** Triangulated surface meshes for the strut and the foil, shown in blue and orange, respectively. The strut mesh is extended in the negative z-direction to allow for the motion of the foil in this direction (color figure online)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

direction, as shown in Fig. 20b. This approach enables us to define the strut geometry as the foil rotates, even though the initial CFD surface mesh does not have any information about the strut geometry after the foil-strut intersection.

Similar to the DLR-F6 configuration, we compute the minimum ratios of area and quality for the surface mesh cells. The changes in minimum ratios as the design is modified are shown in Fig. 22. Figure 23 shows the range of available design changes with the method. This is mainly constrained because of the tight spacing between the foil-strut intersection and the foil trailing edge. As the foil is rotated, the spacing in this region reduces further, and this causes the CFD surface cells on this region to get smaller, limiting the range of the design change. Despite this constraint, the foil geometry can be rotated by about 10° in either direction, which demonstrates the method's robustness even when the clearances between the geometric features are small.

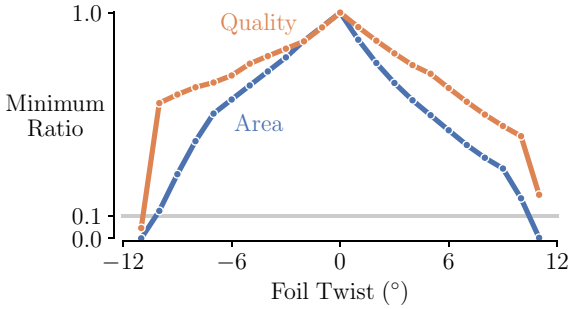


Fig. 22 Minimum ratio of area and quality over each cell as the foil rotates. The minimum ratio metric corresponds to the largest reduction in the area or quality metrics for any cell in the surface mesh compared to the baseline design. We consider a mesh to be a valid surface mesh if both of the minimum ratios for area and quality are above 0.1

3.2 Computational performance

In the context of high-fidelity design optimization, the computational performance of each component in the design framework is essential because hundreds of design iterations may be required to converge to an optimal design. The proposed surface-mesh deformation algorithm contains multiple steps, and the computational performance of each step is crucial for the overall design framework's performance.

There are several computationally demanding steps in the proposed algorithm. Our implementation relies on the pySurf module for the computation of the intersection curve, projection to surfaces and curves, and re-meshing feature curves. In the pySurf module, these operations are implemented using the Fortran 90 language, and the module uses several optimizations to decrease the cost of these computations [12].

Besides these geometric operations, the curve- and point-based deformation approaches introduced in Sects. 2.5 and 2.8 , respectively, require large numbers of operations. For both of these, the displacement of the *seeds* is used to compute a displacement field, which is evaluated at the *evaluation points*, as described in Sects. 2.5 and 2.8 . For the curve-based deformation step (Sect. 2.5), the seeds are defined by the displacements of the line elements that define the intersection and feature curves. Using these seeds, we compute the displacement field at every CFD surface mesh node within $d^\star$ distance from the intersection curve for each component. Similarly, for the point-based deformation step (Sect. 2.8), the seeds are the displacements of the nodes that are projected onto feature curves. Using these, we compute the displacements at the CFD nodes that were originally located on surfaces. For both of these computations, we use weighing functions that are based on the inverse distance of the evaluation points with respect to the seeds as shown in

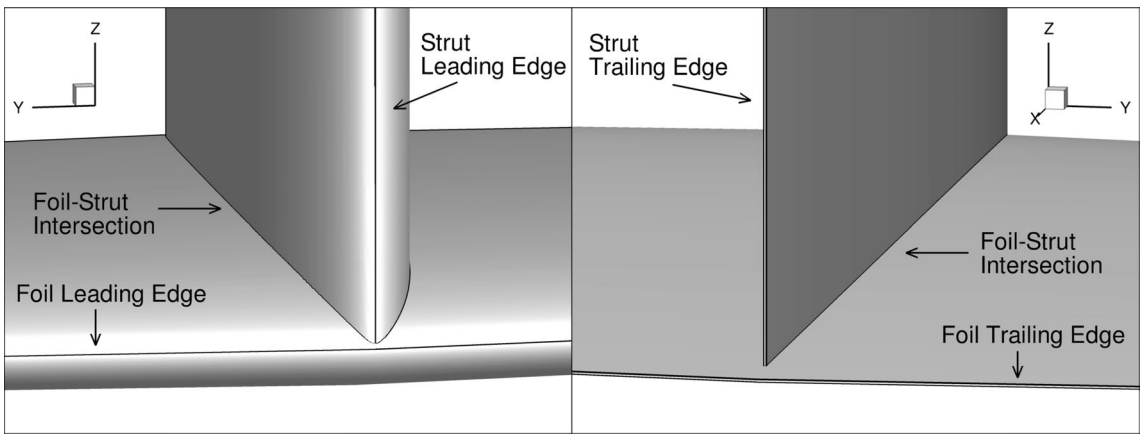


Fig. 21 Intersection and feature curves used for the T-foil geometry. The figure on the left highlights the leading edge, while the figure on the right highlights the configuration's trailing edge. The foil is rotated about the leading edge because of the tight spacing of the leading edge between the foil and the strut

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Eqs. (7) and (13). Both of these terms need to be computed for every combination of seeds and evaluation points.

There are two approaches to compute these weighing terms. The first approach is to compute them once during the first evaluation of the algorithm in an optimization framework and store them for subsequent design iterations. This is possible because both of these weighing functions depend on the seeds and the evaluation points' original locations, so their values do not change. This results in an implementation that has higher memory usage. The second approach is to compute the weighing terms at each design iteration without storing them, which requires less memory usage but more computational effort. In our implementation, we use the second approach. At each design iteration, we compute the weighing terms using vectorized operations in Python.

Since both deformation methods are explicit, the evaluation of the displacement field at a given evaluation point does not depend on the others. As a result, the displacement field can be computed separately for each evaluation point in parallel. The seeds are identical across processors, and each processor only computes the displacement field for the set of evaluation points it contains.

To parallelize the surface mesh deformation method, we simply use the domain decomposition of the CFD solver. The CFD surface mesh is distributed among processors using the domain decomposition of the CFD volume mesh. The triangulated surface mesh is duplicated on each processor in our implementation, but this step can also be parallelized.

Table 1 lists the computational cost of each step in the surface mesh deformation method. For these measurements,

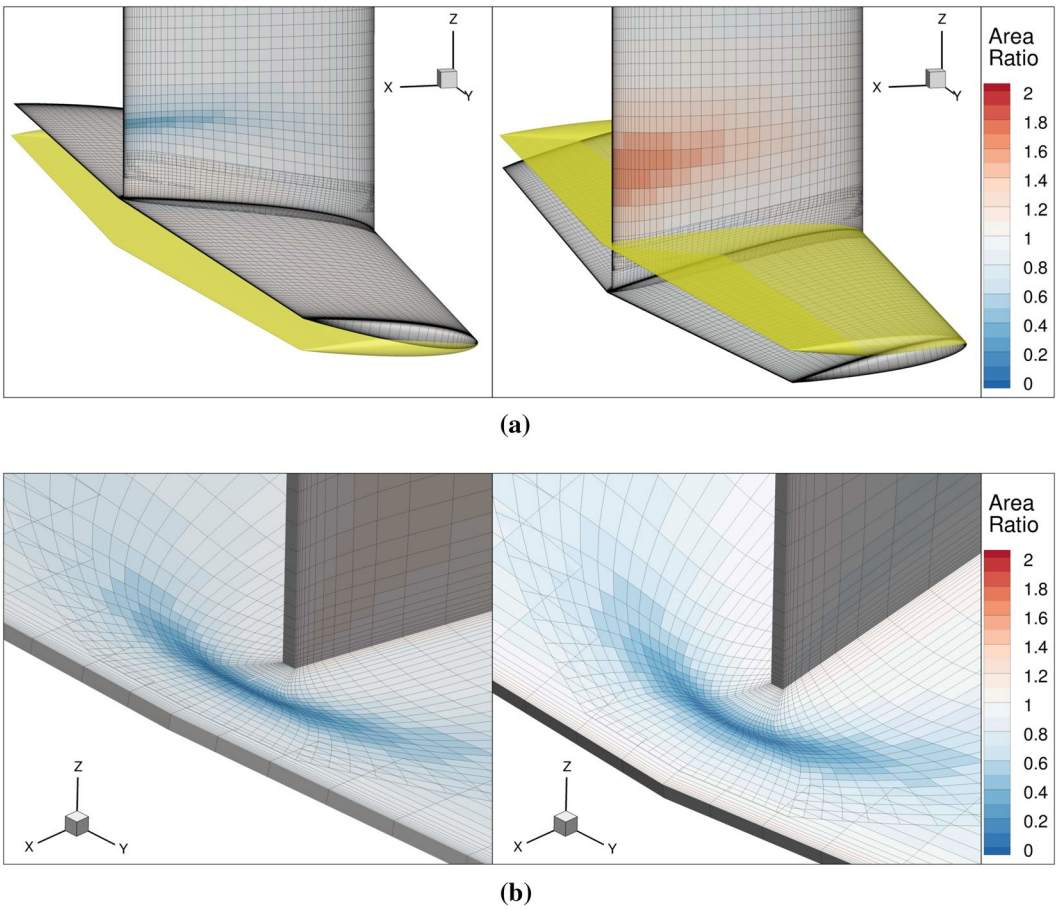


Fig. 23 Range of design changes with the foil rotation. In these figures, the contours show the change in area for each cell compared to the baseline surface mesh. Red regions highlight the cells that have a larger area, and blue regions highlight the cells that have a smaller area after the surface mesh update. **a** Limits of foil rotation. The original foil position is shown in translucent yellow. The minimum (left) and maximum (right) limiting values are equal to − 10.44 and 10.10 degrees, respectively. **b** Detailed view of the trailing edge of the strut-foil intersection for the two limiting design changes presented in **a**. Because the foil is rotated about the leading edge, the trailing edge of the foil moves forward. As a result, the surface cells on the foil geometry near the intersection are compressed, which is the main limiting factor for the range of design changes (color figure online)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

we use the wing-body configuration introduced in Sect. 2.1. The computations are performed on four Skylake nodes of the Electra supercomputer at the NASA Ames Research Center. Each node contains 40 cores, and we use a total of 160 processors for this case. In the timing results for each step of the surface mesh deformation method, we report the slowest processor's timing. The total surface mesh deformation time is measured by synchronizing every processor at the beginning, and the end of the surface mesh update and measuring the time between after each synchronization point. Therefore, the total reported time can be less than the sum of individual step timings. This is because there are few collective communications required for the method, and the slowest processor is not guaranteed to be the same across every step of the method.

Besides the cost of the algorithm, we also report the computational cost of deforming the volume mesh and performing one CFD simulation using the resulting mesh in Table 1. The flow conditions and the computational framework used for this case are detailed in Sect. 4, and these performance metrics are representative of one design evaluation. Using these results, we can see that the relative cost of the surface-mesh deformation algorithm is around 2% of the computational effort of one CFD simulation.

Because of the low relative cost of the surface mesh deformation method compared to the CFD simulations, we did not include any additional code optimizations. However, the implementation could be further optimized in a number of ways. First, instead of using vectorized operations in Python, which is an interpreted programming language, the curve and point-based deformation steps could be performed using a compiled programming language such as Fortran 90. Second, a better parallelization approach could be used. Our implementation uses the domain decomposition of the CFD solver, but this is not guaranteed to yield a perfect load balancing between processors. This is because while the CFD solver does try to balance the surface mesh distribution across processors in addition to balancing the number of volume cells, the surface mesh nodes near intersecting components require many additional computations compared to the surface mesh nodes away from the intersections. As a result, while the total number of surface mesh nodes might be well balanced, the total number of nodes for which the intersection treatment is applied might not be equally distributed among processors. This can be solved by performing another load balancing step for the nodes that are within $d^\star$ distance from the intersections. Finally, the curve and point-based deformation methods can be modified in a similar way as described by Luke et al. [43]. With this approach, for each evaluation point, the displacement seeds far away from the evaluation point can be aggregated, which reduces the total number of operations required for each evaluation point.

Table 1 Wall time required for each step of the surface mesh algorithm, along with the wall time of the volume mesh deformation and CFD simulation

Section	Step	Wall time (s)
2.2	Component-based design changes	0.103
2.3	Intersection curve computation	0.578
2.4	Feature curve computation	0.027
2.5	Curve-based deformation	0.368
2.6	Blending the two displacement fields	–
2.7	Projection to curves	0.005
2.8	Point-based deformation	0.031
2.9	Projection to surfaces	0.689
	Total surface mesh deformation	1.762
	Volume mesh deformation	0.822
	CFD simulation	107.533

The timing for the blending step (Sect. 2.6) is not included because these operations are included in the curve-based deformation step (Sect. 2.5). The surface mesh contains 21,080 surface cells and the volume mesh contains about 1.1 million volume cells. The surface mesh algorithm cost is comparable to the volume mesh deformation algorithm cost, and the surface and volume mesh deformation costs are about 1.6% and 0.8% of the CFD simulation cost, respectively

4 Optimization of a wing-body junction

To demonstrate the proposed method's capabilities in the context of an aerodynamic shape optimization (ASO) problem, we optimize the wing-fuselage intersection of the DLR-F6 configuration. The baseline design has a large area of flow separation on the upper skin of the wing near the fuselage at the cruise condition. There are several drag computation studies for this configuration [46, 57, 58], along with efforts to design a wing-fairing to reduce the amount of separation near the fuselage intersection [35, 36, 38, 59].

In this work, we optimize the wing-fuselage intersection of the DLR-F6 configuration using the proposed surface mesh deformation method. With this ASO study, we only modify the fuselage design and do not change the wing design. This study is not meant to be a benchmark for the ASO of the wing-fuselage intersection of the DLR-F6 configuration. Our only goal is to demonstrate the effectiveness of the surface mesh deformation approach.

4.1 Computational framework

The ASO framework we use in this work is MACH-Aero. This framework is a subset of tools available in the MDO of aircraft configurations with high fidelity (MACH) framework, which was developed for aerostructural design optimization [60]. MACH-Aero is an open-source, high-performance optimization framework that has been extensively used for high-fidelity aerodynamic shape optimization

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Optimization problem definition

	Variable/Function	Description	Quantity
Minimize	C_D	Drag coefficient	1
With respect to	$-4° \leq \alpha \leq 10°$	Angle of attack	1
	$0\,\text{mm} \leq n_{\text{Fuselage}} \leq 40\,\text{mm}$	Perturbation to the fuselage FFD points	16
		Total	17
Subject to	$C_L = 0.5$	Lift coefficient at cruise	1

studies [4, 5, 61, 62].[2] In this optimization study, we use the FFD approach to define component-based design variables [15]. To define design variables for the fuselage geometry, we use the 16 FFD points introduced in Fig. 17a. This design parametrization is identical to that of Secco et al. [38].

At each design iteration, we use the surface mesh deformation method proposed herein to obtain an updated CFD surface mesh. Then, we use the mesh deformation approach described by Secco et al. [44] to propagate the displacements of the surface mesh to the volume mesh. To analyze the design at each iteration, we use ADflow, which is an open-source CFD solver that can solve RANS equations on structured overset meshes [63].[3] The adjoint implementation in ADflow enables us to evaluate the derivatives of functions of interest with respect to design variables efficiently [64]. Finally, we use SNOPT [65] as the optimizer for this study, through the Python interface pyOptSparse [66].[4] The optimization study was performed using four Skylake nodes on the Electra supercomputer at the NASA Ames Research Center.

4.2 Optimization problem definition

The optimization problem minimizes the drag coefficient (C_D) of the DLR-F6 configuration at the nominal cruise condition (Mach number of 0.75 and a Reynolds number of 5×10^6). We include a constraint on the lift coefficient such that $C_L = 0.5$ and an angle of attack design variable. The 16 FFD control points on the fuselage FFD volume (Fig. 3a) are used to parametrize the fuselage geometry near the wing-fuselage intersection. The control points are allowed to individually move radially outward from the fuselage center to prevent shrinking the fuselage design compared to the original surface [47]. Table 2 details the optimization problem, along with the design variable bounds.

The CFD volume mesh we use is the coarse level mesh used by Secco [12], and it contains about 1.1 million volume cells. Similarly to previous work [12, 38], we use the SA-R-QCR2000 turbulence model [67–69] for the RANS formulation. At each design iteration, we converge the CFD solver residuals by 12 orders of magnitude relative to the free-stream residual norm using the approximate Newton–Krylov solver in ADflow [70]. Similarly, we converge the adjoint solution by the same relative amount. For the surface mesh deformation method, we use the same configuration and triangulated surface meshes we used for the example design changes in Sect. 2.

4.3 Optimization results

The optimization converged in 29 iterations with a feasibility of 5.3×10^{-8}. The initial and final optimality values are 7.5×10^{-3} and 6.9×10^{-6}, respectively. The final feasibility and optimality metrics are below the prescribed tolerance of 10^{-5}. The baseline and optimized designs result in 290.59 and 284.19 counts of drag at $C_L = 0.5$ (one drag count corresponds to $C_D \times 10^4$). This is equivalent to a 2.2% reduction in C_D. This result is not meant to be a benchmark value for the DLR-F6 optimizations, and it is reported to demonstrate the capabilities of the surface mesh deformation method in an optimization framework. Compared to previous optimizations performed by Secco [12], there are several improvements in the framework responsible for the lower drag obtained herein.

The optimizer perturbs the fuselage design, which results in a modified intersection curve between the wing and the fuselage (Fig. 24). The surface mesh on both the fuselage and the wing is modified to accommodate the changing intersection curve; however, the wing geometry is identical to the baseline geometry. The optimizer successfully removes the separation region near the intersection, which is the primary source of reduction in drag (Fig. 25). This result demonstrates the effectiveness of the method in an ASO framework.

5 Conclusions

This paper introduced a surface mesh deformation method that can update the CFD surface meshes near component intersections. The method consists of the eight steps summarized in Sect. 2.10. We demonstrated each step using

[2] https://github.com/mdolab/MACH-Aero, Accessed December 2020.

[3] https://github.com/mdolab/adflow, Accessed December 2020.

[4] https://github.com/mdolab/pyoptsparse, Accessed December 2020.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

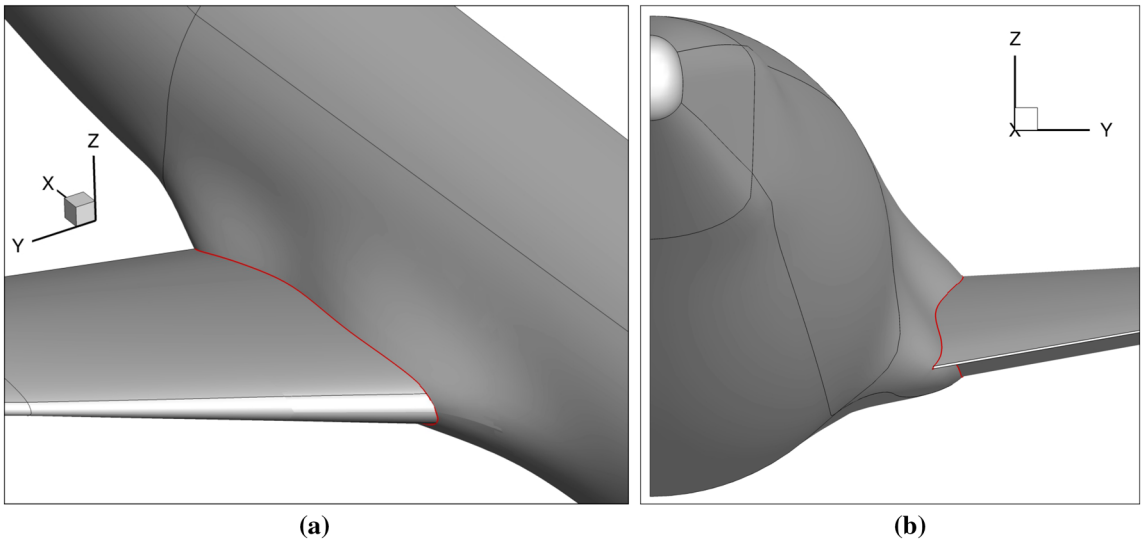


Fig. 24 Optimized geometry with updated intersection curve highlighted in red. **a** Top-front view. **b** Rear view (color figure online)

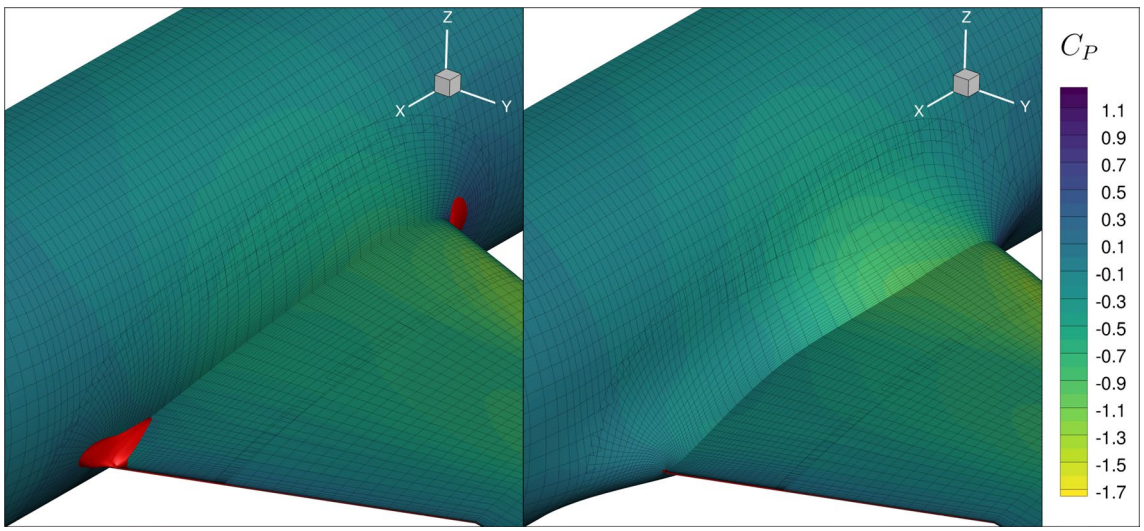


Fig. 25 Pressure coefficient distributions of the baseline (left) and optimized (right) geometries. The region where the flow is separated is highlighted in red. The optimizer removed separation near the wing-fuselage intersection by perturbing the fuselage geometry (color figure online)

an example configuration that is representative of many aircraft configurations. Compared to previous approaches, the method smoothly deforms the CFD surface mesh without introducing any topological changes to the overset mesh. This prevents difficulties that can arise because of the numerical noise caused by these topological changes, making it suitable to use within ASO frameworks. Using two example configurations, we demonstrated the range of design changes obtained with the method and studied the ASO of a wing-fuselage intersection. The method applies to both structured and unstructured CFD meshes and makes it possible to use CAD and CDG tools in high-fidelity design optimization.

Appendix A: Computation of the curve-based deformation

The curve-based deformation step introduced in Sect. 2.5 relies on the computation of two integrals in the nominator and denominator in Eq. (6). The equation can be written again using a simplified notation for a general case as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

$$\Delta\mathbf{x} = \frac{\int_{\mathcal{C}} w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t}{\int_{\mathcal{C}} w(\mathbf{x},t)L\mathrm{d}t}, \tag{16}$$

where $\mathcal{C}$ represents the original location of the curve and $\Delta\mathcal{C}$ is the displacement defined on this curve, such that $\mathcal{C}+\Delta\mathcal{C}$ yields the updated coordinates of the curve. Using these displacements, the method computes the displacement of a given point $\mathbf{x}$, using a weighted averaging based on the inverse distance of the point to the curve. The parameter $t \in [0,1]$ is used to traverse the curve $\mathcal{C}$ with length L and the weight w can be written as

$$w(\mathbf{x},t) = \frac{1}{||\mathbf{x}-\mathcal{C}(t)||_2^m}, \tag{17}$$

where m is an exponent, which set to 3 in this work. In the next two sections, we inspect the behavior of this approach for the special case when the evaluation point $\mathbf{x}$ is located on the curve $\mathcal{C}$ and detail the computation of the terms for this method when the curve $\mathcal{C}$ is defined using line elements.

A.1: Special case for points located on curves

The weighting function has a finite value as long as the evaluation point $\mathbf{x}$ is not on the curve $\mathcal{C}$. For the special case where the evaluation point is on the curve and $m > 0$, the weighing function tends to infinity as the parameter $t \to t_*$, where the evaluation point $\mathbf{x}$ is co-located with the point $\mathcal{C}(t_*)$. This can be written as

$$\lim_{t\to t_*} w(\mathbf{x},t) = \lim_{t\to t_*} \frac{1}{||\mathbf{x}-\mathcal{C}(t)||_2^m} \to \infty. \tag{18}$$

As a result, the two integrands we define also tend to infinity as $t \to t_*$, which results in two improper integrals.

The integrations over the curve $\mathcal{C}$, which is defined in $t = [0,1]$, can be split into two at $t = T$, which results in

$$\Delta\mathbf{x} = \frac{\int_0^T w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t + \int_T^1 w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t}{\int_0^T w(\mathbf{x},t)L\mathrm{d}t + \int_T^1 w(\mathbf{x},t)L\mathrm{d}t}. \tag{19}$$

Then, we substitute $t = -s$ for the two integrals that go from T to 1, and reverse the integration limits, which yields

$$\Delta\mathbf{x} = \frac{\int_0^T w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)\Delta\mathcal{C}(-s)L\mathrm{d}s}{\int_0^T w(\mathbf{x},t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)L\mathrm{d}s}. \tag{20}$$

For the special case where $\mathbf{x}$ is co-located with $\mathcal{C}(t_*)$ and $m \geq 1$, these integrals do not converge; however, $\Delta\mathbf{x}$ can be computed using the L'Hôpital's rule as

$$\begin{aligned}
\Delta\mathbf{x} &= \lim_{T\to t_*^-} \frac{\int_0^T w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)\Delta\mathcal{C}(-s)L\mathrm{d}s}{\int_0^T w(\mathbf{x},t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)L\mathrm{d}s} \\
&= \lim_{T\to t_*^-} \frac{\frac{\mathrm{d}}{\mathrm{d}T}\left(\int_0^T w(\mathbf{x},t)\Delta\mathcal{C}(t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)\Delta\mathcal{C}(-s)L\mathrm{d}s\right)}{\frac{\mathrm{d}}{\mathrm{d}T}\left(\int_0^T w(\mathbf{x},t)L\mathrm{d}t + \int_{-1}^{-T} w(\mathbf{x},-s)L\mathrm{d}s\right)} \\
&= \lim_{T\to t_*^-} \frac{2w(\mathbf{x},T)\Delta\mathcal{C}(T)L}{2w(\mathbf{x},T)L} \\
&= \Delta\mathcal{C}(t_*).
\end{aligned} \tag{21}$$

This result means that when $\mathbf{x}$ is initially co-located with $\mathcal{C}(t_*)$, the displacement $\Delta\mathbf{x}$ computed using Eq. (16) results in the displacement of the curve at $t = t_*$, which is the desired behavior demonstrated in Fig. 10. With this approach, the surface mesh nodes that are initially located on the intersection and feature curves directly follow the displacements of these curves.

A.2: Computation of the integrals over piecewise linear curves

In our application, the curve $\mathcal{C}$ is defined using a union of line elements. Using this definition, the two integrals introduced in Eq. (16) can be computed as a sum over every line element, where we compute the integral for each line element separately.

Without loss of generality, we consider the case where the curve $\mathcal{C}$ is defined using a single line element, with endpoints $\mathbf{x}_A$ and $\mathbf{x}_B$. In this case, we use the parameter $t \in [0,1]$ to traverse this line element, and the coordinate along this line element can be written as

$$\mathcal{C}(t) = \mathbf{x}_A(1-t) + \mathbf{x}_B t, \tag{22}$$

such that $\mathcal{C}(0) = \mathbf{x}_A = (x_A, y_A, z_A)$ and $\mathcal{C}(1) = \mathbf{x}_B = (x_B, y_B, z_B)$ in Cartesian coordinates. Similarly, we can write the displacement of the curve as

$$\Delta\mathcal{C}(t) = \Delta\mathbf{x}_A(1-t) + \Delta\mathbf{x}_B t, \tag{23}$$

where $\Delta\mathbf{x}_A$ and $\Delta\mathbf{x}_B$ are the displacements of the points that define the line element.

Using these definitions, we can re-write $\Delta\mathbf{x}$ as

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

$$\Delta\mathbf{x} = \frac{\Delta\mathbf{x}_A \int_0^1 w(\mathbf{x},t)L\mathrm{d}t + (\Delta\mathbf{x}_B - \Delta\mathbf{x}_A)\int_0^1 w(\mathbf{x},t)Lt\mathrm{d}t}{\int_0^1 w(\mathbf{x},t)L\mathrm{d}t}. \quad (24)$$

The terms $\Delta\mathbf{x}_A$ and $\Delta\mathbf{x}_B$ correspond to the displacements defined at the two endpoints of the line and can be taken out of the integrals because they do not depend on the parameter t. This expression can be evaluated by evaluating the two integrals that appear, which can be defined as

$$I_1 = \int_0^1 w(\mathbf{x},t)L\mathrm{d}t, \quad I_2 = \int_0^1 w(\mathbf{x},t)Lt\mathrm{d}t. \quad (25)$$

In this work, $m = 3$ in the definition of w (17); therefore, we specialize the exact integration of these terms for this value. In this case, we can write the two integrals as

$$I_1 = \int_0^1 \frac{L}{||\mathbf{x} - \mathcal{C}(t)||_2^3}\mathrm{d}t = \int_0^1 \frac{L}{\left|\left|\mathbf{x} - \left(\mathbf{x}_A(1-t) + \mathbf{x}_B t,\right)\right|\right|_2^3}\mathrm{d}t,$$
$$I_2 = \int_0^1 \frac{Lt}{||\mathbf{x} - \mathcal{C}(t)||_2^3}\mathrm{d}t = \int_0^1 \frac{Lt}{\left|\left|\mathbf{x} - \left(\mathbf{x}_A(1-t) + \mathbf{x}_B t,\right)\right|\right|_2^3}\mathrm{d}t. \quad (26)$$

By expressing the evaluation point $\mathbf{x}$ as (x, y, z) in Cartesian coordinates, the 2-norm that appears in the denominator of both integrals can be written as

$$\left|\left|\mathbf{x} - \left(\mathbf{x}_A(1-t) + \mathbf{x}_B t,\right)\right|\right|_2 = (at^2 + bt + c)^{1/2}, \quad (27)$$

where

$$a = L^2 = (x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2,$$
$$b = 2((x_B - x_A)(x_A - x) + (y_B - y_A)(y_A - y) + (z_B - z_A)(z_A - z)),$$
$$c = (x_A - x)^2 + (y_A - y)^2 + (z_A - z)^2. \quad (28)$$

With these definitions, the two integrals can be analytically computed as

$$I_1 = -\frac{2(2at+b)\sqrt{a}}{(b^2-4ac)\sqrt{t(at+b)+c}}\Bigg|_0^1 = -\frac{2(2a+b)\sqrt{a}}{(b^2-4ac)\sqrt{a+b+c}} + \frac{2b\sqrt{a}}{(b^2-4ac)\sqrt{c}},$$
$$I_2 = \frac{2(2bt+c)\sqrt{a}}{(b^2-4ac)\sqrt{t(at+b)+c}}\Bigg|_0^1 = \frac{2(2b+c)\sqrt{a}}{(b^2-4ac)\sqrt{a+b+c}} - \frac{2c\sqrt{a}}{(b^2-4ac)\sqrt{c}}. \quad (29)$$

In this formulation, the terms $a+b+c$ and c can have small negative values because of machine precision errors. Therefore, when computing the square roots of these terms, we set them to zero if they are less than zero to prevent results with nonzero imaginary parts. Finally, the terms $(b^2-4ac)\sqrt{a+b+c}$ and $(b^2-4ac)\sqrt{c}$ can have a value of zero. This is problematic because these terms appear in the denominators. Therefore, we subtract a small number $\epsilon = 10^{-50}$ from the denominators. We subtract ϵ instead of adding it, because the result of $\sqrt{a+b+c}$ and $\sqrt{c}$ are guaranteed to be non-negative, while the term $b^2 - 4ac \leq 0$. With these considerations, we can robustly compute the terms required for the curve-based displacement approach introduced in Eq. (6).

Acknowledgements This work was funded by the NASA Advanced Air Transport Technology (AATT) and Transformational Tools and Technologies (TTT) projects. Computational resources were provided by the NASA High-End Computing (HEC) Program through the NASA Advanced Supercomputing (NAS) Division at Ames Research Center, and the Extreme Science and Engineering Discovery Environment (XSEDE), which is supported by the National Science Foundation grant number ACI-1548562. We are especially grateful to Justin Gray for his guidance and support. Thanks to Ney Secco for developing the pySurf framework, Yingqian Liao for providing the T-shaped hydrofoil geometry, Benjamin Brelje for reviewing an early version of this paper, and Gaetan Kenway for developing an earlier version of the surface-mesh deformation method that led to this work.

References

1. Jameson A (1988) Aerodynamic design via control theory. J Sci Comput 3(3):233–260. https://doi.org/10.1007/BF01061285
2. Kenway GKW, Martins JRRA (2014) Multipoint high-fidelity aerostructural optimization of a transport aircraft configuration. J Aircr 51(1):144–160. https://doi.org/10.2514/1.C032150
3. Carrier G, Destarac D, Dumont A, Méheut M, Din Itham SE, Jacques P, Saloua Ben K, Brezillon J, Pestana M (2014) Gradient-based aerodynamic optimization with the *elsA* software. In: 52nd Aerospace Sciences Meeting. https://doi.org/10.2514/6.2014-0568
4. Lyu Z, Kenway GKW, Martins JRRA (2015) Aerodynamic shape optimization investigations of the Common Research Model wing benchmark. AIAA J 53(4):968–985. https://doi.org/10.2514/1.J053318
5. Kenway GKW, Martins JRRA (2016) Multipoint aerodynamic shape optimization investigations of the Common Research Model wing. AIAA J 54(1):113–128. https://doi.org/10.2514/1.J054154
6. Kenway GKW, Martins JRRA (2017) Buffet onset constraint formulation for aerodynamic shape optimization. AIAA J 55(6):1930–1947. https://doi.org/10.2514/1.J055172
7. Bons NP, Mader CA, Martins JRRA, Cuco APC, Odaguil FIK (2018) High-fidelity aerodynamic shape optimization of a full configuration regional jet. In: 2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference, Kissimmee, FL. https://doi.org/10.2514/6.2018-0106
8. Reist TA, Koo D, Zingg DW, Bochud P, Castonguay P, Leblond D (2020) Cross validation of aerodynamic shape optimization methodologies for aircraft wing-body optimization. AIAA J. https://doi.org/10.2514/1.J059091

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

9. Chan W, Meakin R (1997) Advances towards automatic surface domain decomposition and grid generation for overset grids. In: 13th Computational Fluid Dynamics Conference, fluid dynamics and co-located conferences. american institute of aeronautics and astronautics. https://doi.org/10.2514/6.1997-1979
10. Chan W, Gomez R, Rogers S, Buning P (2002) Best practices in overset grid generation. In: 32nd AIAA Fluid Dynamics Conference and Exhibit, Fluid Dynamics and Co-located Conferences. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2002-3191
11. Kleb WL, Park MA, Wood WA, Bibb KL, Thompson KB, Gomez RJ (2019) Sketch-to-solution: An exploration of viscous CFD with automatic grids. In: AIAA Aviation Forum, AIAA AVIATION Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2019-2948
12. Secco N (2018) Component-based aerodynamic shape optimization using overset meshes. PhD thesis, University of Michigan, Ann Arbor, MI. https://deepblue.lib.umich.edu/handle/2027.42/146042. Accessed 6 Jan 2021
13. Samareh JA (2001) Survey of shape parameterization techniques for high-fidelity multidisciplinary shape optimization. AIAA J 39(5):877–884. https://doi.org/10.2514/2.1391
14. Zhang T-T, Wang Z-G, Huang W, Yan L (2018) A review of parametric approaches specific to aerodynamic design process. Acta Astronaut 145:319–331. https://doi.org/10.1016/j.actaastro.2018.02.011 (**ISSN 0094-5765**)
15. Kenway GKW, Kennedy GJ, Martins JRRA (2010) A CAD-free approach to high-fidelity aerostructural optimization. In: Proceedings of the 13th AIAA/ISSMO Multidisciplinary Analysis Optimization Conference, number AIAA 2010-9231, Fort Worth, TX. https://doi.org/10.2514/6.2010-9231
16. Gagnon H, Zingg DW (2015) Two-level free-form and axial deformation for exploratory aerodynamic shape optimization. AIAA J 53(7):2015–2026. https://doi.org/10.2514/1.J053575 (**ISSN 0001-1452**)
17. Taylor NJ (2015) Industrial perspectives on geometry handling for aerodynamics. In: 22nd AIAA Computational Fluid Dynamics Conference, AIAA Aviation Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2015-3408
18. Haimes R, Drela M (2012) On the construction of aircraft conceptual geometry for high-fidelity analysis and design. In: 50th AIAA Aerospace Sciences Meeting including the New Horizons Forum and Aerospace Exposition, Aerospace Sciences Meetings. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2012-683
19. Haimes R, Dannenhoffer J (2013) The Engineering Sketch Pad: A solid-modeling, feature-based, web-enabled system for building parametric geometry. In: 21st AIAA Computational Fluid Dynamics Conference, Fluid Dynamics and Co-located Conferences. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2013-3073
20. Mukundakrishnan B, Rajmohan N, Rajnarayan DG, Fugal S (2019) A script-based CAD system for aerodynamic design. In: AIAA Aviation Forum, AIAA AVIATION Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2019-3069
21. Hwang JT, Martins JRRA (2012) GeoMACH: Geometry-centric MDAO of aircraft configurations with high fidelity. In: Proceedings of the 14th AIAA/ISSMO Multidisciplinary Analysis Optimization Conference, Indianapolis, IN. https://doi.org/10.2514/6.2012-5605
22. Hahn A (2010) Vehicle Sketch Pad: A parametric geometry modeler for conceptual aircraft design. In: 48th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, Aerospace Sciences Meetings. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2010-657
23. McDonald RA (2015) Interactive reconstruction of 3D models in the OpenVSP parametric geometry tool. In: 53rd AIAA Aerospace Sciences Meeting, AIAA SciTech Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2015-1014
24. Alonso JJ, Martins JRRA, Reuther JJ, Haimes R (2003) High-fidelity aero-structural design using a parametric CAD-based model. In: Proceedings of the 16th AIAA Computational Fluid Dynamics Conference, Orlando, FL. https://doi.org/10.2514/6.2003-3429
25. Bobrowski K, Ferrer E, Valero E, Barnewitz H (2017) Aerodynamic shape optimization using geometry surrogates and adjoint method. AIAA J 55(10):3304–3317. https://doi.org/10.2514/1.J055766
26. Heath C, Slater JW, Rallabhandi SK (2016) Inlet trade study for a low boom aircraft demonstrator. In: 34th AIAA Applied Aerodynamics Conference, AIAA Aviation Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2016-4050
27. Dannenhoffer J, Haimes R (2015) Design sensitivity calculations directly on CAD-based geometry. In: 53rd AIAA Aerospace Sciences Meeting, AIAA SciTech Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2015-1370
28. Banović M, Mykhaskiv O, Auriemma S, Walther A, Legrand Herve, Müller Jens-Dominik (2018) Algorithmic differentiation of the Open CASCADE Technology CAD kernel and its coupling with an adjoint CFD solver. Optim Methods Softw 33(4–6):813–828. https://doi.org/10.1080/10556788.2018.1431235 (**ISSN 1055-6788**)
29. Agarwal D, Robinson TT, Armstrong CG, Marques S, Vasilopoulos I, Meyer M (2018) Parametric design velocity computation for CAD-based design optimization using adjoint methods. Eng Comput 34(2):225–239. https://doi.org/10.1007/s00366-017-0534-x (**ISSN 1435-5663**)
30. Shenren X, Jahn W, Müller J-D (2014) CAD-based shape optimisation with CFD using a discrete adjoint. Int J Numer Methods Fluids 74(3):153–168. https://doi.org/10.1002/fld.3844
31. Truong AH, Zingg DW, Haimes R (2016) Surface mesh movement algorithm for computer-aided-design-based aerodynamic shape optimization. AIAA J 54(2):542–556. https://doi.org/10.2514/1.J054295 (**ISSN 0001-1452**)
32. Sun L, Yao W, Robinson T, Marques S, Armstrong C (2020) A framework of gradient-based shape optimization using feature-based CAD parameterization. In: AIAA Scitech 2020 Forum, Orlando, FL. American Institute of Aeronautics and Astronautics. ISBN 978-1-62410-595-1. https://doi.org/10.2514/6.2020-0889
33. Kim H-J, Nakahashi K (2007) Surface mesh movement for aerodynamic design of body-installation junction. AIAA J 45(5):1138–1142. https://doi.org/10.2514/1.12947 (**ISSN 0001-1452**)
34. Koc S, Kim H-J, Nakahashi K (2006) Aerodynamic design of complex configurations with junctions. J Aircr 43(6):1838–1844. https://doi.org/10.2514/1.20723
35. Brezillon J, Dwight RP (2012) Applications of a discrete viscous adjoint method for aerodynamic shape optimisation of 3D configurations. CEAS Aeronaut J 3(1):25–34. https://doi.org/10.1007/s13272-011-0038-0 (**ISSN 1869-5582**)
36. Xu S, Timme S, Mykhaskiv O, Müller J-D (2017) Wing-body junction optimisation with CAD-based parametrisation including a moving intersection. Aerosp Sci Technol 68:543–551. https://doi.org/10.1016/j.ast.2017.06.014 (**ISSN 1270-9638**)
37. Mykhaskiv O, Mohanamuraly P, Mueller J-D, Xu S, Timme S (2017) CAD-based shape optimisation of the NASA CRM wing-body intersection using differentiated CAD-kernel. In: 35th AIAA Applied Aerodynamics Conference, AIAA AVIATION Forum. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2017-4080
38. Secco NR, Jasa JP, Kenway GKW, Martins JRRA (2018) Component-based geometry manipulation for aerodynamic shape

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

optimization with overset meshes. AIAA J 56(9):3667–3679. https://doi.org/10.2514/1.J056550
39. Secco NR, Martins JRRA (2019) RANS-based aerodynamic shape optimization of a strut-braced wing with overset meshes. J Aircr 56(1):217–227. https://doi.org/10.2514/1.C034934
40. de Boer A, van der Schoot MS, Bijl H (2007) Mesh deformation based on radial basis function interpolation. Comput Struct 85(11):784–795. https://doi.org/10.1016/j.compstruc.2007.01.013 (**ISSN 0045-7949**)
41. Coulier P, Darve E (2016) Efficient mesh deformation based on radial basis function interpolation by means of the inverse fast multipole method. Comput Methods Appl Mech Eng 308:286–309. https://doi.org/10.1016/j.cma.2016.05.029 (**ISSN 0045-7825**)
42. Witteveen J, Bijl H (2009) Explicit mesh deformation using inverse distance weighting interpolation. In: 19th AIAA Computational Fluid Dynamics, Fluid Dynamics and Co-located Conferences. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2009-3996
43. Luke E, Collins E, Blades E (2012) A fast mesh deformation method using explicit interpolation. J Comput Phys 231(2):586–601. https://doi.org/10.1016/j.jcp.2011.09.021 (**ISSN 0021-9991**)
44. Secco N, Kenway GKW, He P, Mader CA, Martins JRRA (2020) Efficient mesh generation and deformation for aerodynamic shape optimization. AIAA J (**In press**)
45. He X, Li J, Mader CA, Yildirim A, Martins JRRA (2019) Robust aerodynamic shape optimization–from a circle to an airfoil. Aerosp Sci Technol 87:48–61. https://doi.org/10.1016/j.ast.2019.01.051
46. Brodersen O (2002) Drag prediction of engine-airframe interference effects using unstructured Navier-Stokes calculations. J Aircr 39(6):927–935. https://doi.org/10.2514/2.3037
47. Secco NR, Jasa JP, Kenway GKW, Martins JRRA (2017) Component-based geometry manipulation for aerodynamic shape optimization with overset meshes. In: 18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2017-3327
48. Patrikalakis NM (1993) Surface-to-surface intersections. IEEE Comput Graph Appl 13(1):89–95. https://doi.org/10.1109/38.180122 (**ISSN 0272-1716**)
49. Kenway GKW, Secco N, Martins JRRA, Mishra A, Duraisamy K (2017) An efficient parallel overset method for aerodynamic shape optimization. In: Proceedings of the 58th AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference, AIAA SciTech Forum, Grapevine, TX. https://doi.org/10.2514/6.2017-0357
50. Shepard D (1968) A two-dimensional interpolation function for irregularly-spaced data. In: Proceedings of the 1968 23rd ACM National Conference, ACM '68, pages 517–524, New York, NY, USA. ACM. https://doi.org/10.1145/800186.810616
51. Mark A (2013) Finlayson. ANSYS ICEM CFD User's Manual. ANSYS Inc, Canonsburg
52. Liao Y, Yildirim A, Lu YY, Martins JRRA (2020) Hydrodynamic optimization of a T-foil. In: SNAME Maritime Convention. The Society of Naval Architects and Marine Engineers. https://www.onepetro.org/conference-paper/SNAME-SMC-2020-084. Accessed 6 Jan 2021
53. Binns JR, Brandner PA, Plouhinec J (2008) The effect of heel angle and free-surface proximity on the performance and strut wake of a moth sailing dinghy rudder T-foil. Auckland, NZ. https://eprints.utas.edu.au/8987/. Accessed 6 Jan 2021
54. Sacher M, Durand M, Élisa B, Hauville F, Duvigneau R, Le Maître O, Astolfi J-A (2018) Flexible hydrofoil optimization for the 35th America's Cup with constrained EGO method. Ocean Eng 157:62–72. https://doi.org/10.1016/j.oceaneng.2018.03.047 (**ISSN 0029-8018**)
55. Wackers J, Guilmineau E, Palmieri A, Queutey P (2014) Hessian-based grid refinement for the simulation of surface-piercing hydrofoils. In: 17th Numerical Towing Tank Symposium (NuTTS 2014), Marstrand, Sweden. https://hal.archives-ouvertes.fr/hal-01202486. Accessed 6 Jan 2021
56. Barden TA, Binns JR (2012) On the road to establishing ventilation probability for Moth sailing dinghies. Australasian Fluid Mechanics Society. https://trove.nla.gov.au/version/191926247. Accessed 6 Jan 2021
57. Laflin KR, Klausmeyer SM, Zickuhr T, Vassberg JC, Wahls RA, Morrison JH, Brodersen OP, Rakowitz ME, Tinoco EN, Godard J-L (2005) Data summary from second AIAA computational fluid dynamics drag prediction workshop. J Aircr 42(5):1165–1178. https://doi.org/10.2514/1.10771
58. Vassberg J, Tinoco E, Mani M, Brodersen O, Eisfeld B, Wahls R, Morrison J, Zickuhr T, Laflin K, Mavriplis D (2007) Summary of the third AIAA CFD drag prediction workshop. In: 45th AIAA Aerospace Sciences Meeting and Exhibit, Aerospace Sciences Meetings. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2007-260
59. Vassberg J, Sclafani A, DeHaan M (2005) A wing-body fairing design for the DLR-F6 model: A DPW-III case study. In: 23rd AIAA Applied Aerodynamics Conference, Fluid Dynamics and Co-located Conferences. American Institute of Aeronautics and Astronautics. https://doi.org/10.2514/6.2005-4730
60. Kenway GKW, Kennedy GJ, Martins JRRA (2014) Scalable parallel approach for high-fidelity steady-state aeroelastic analysis and derivative computations. AIAA J 52(5):935–951. https://doi.org/10.2514/1.J052255
61. Lyu Z, Martins JRRA (2015) Aerodynamic shape optimization of an adaptive morphing trailing edge wing. J Aircr 52(6):1951–1970. https://doi.org/10.2514/1.C033116
62. Chen S, Lyu Z, Kenway GKW, Martins JRRA (2016) Aerodynamic shape optimization of the Common Research Model wing-body-tail configuration. J Aircr 53(1):276–293. https://doi.org/10.2514/1.C033328
63. Mader CA, Kenway GKW, Yildirim A, Martins JRRA (2020) ADflow: An open-source computational fluid dynamics solver for aerodynamic and multidisciplinary optimization. J Aerosp Inf Syst. https://doi.org/10.2514/1.I010796
64. Kenway GKW, Mader CA, He P, Martins JRRA (2019) Effective adjoint approaches for computational fluid dynamics. Progress Aerosp Sci 110:100542. https://doi.org/10.1016/j.paerosci.2019.05.002
65. Gill PE, Murray W, Saunders MA (2002) SNOPT: an SQP algorithm for large-scale constrained optimization. SIAM J Optim 12(4):979–1006. https://doi.org/10.1137/S1052623499350013
66. Neil W, Kenway G, Mader CA, Jasa J, Martins JRRA (2020) pyOptSparse: A Python framework for large-scale constrained nonlinear optimization of sparse systems. J Open Source Softw 5(54):2564. https://doi.org/10.21105/joss.02564
67. Spalart P, Allmaras S (1994) A one-equation turbulence model for aerodynamic flows. La Recherche Aerosp 1:5–21
68. Dacles-Mariani J, Zilliac GG, Chow JS, Bradshaw P (1995) Numerical/experimental study of a wingtip vortex in the near field. AIAA J 33(9):1561–1568. https://doi.org/10.2514/3.12826
69. Spalart PR (2000) Strategies for turbulence modelling and simulations. Int J Heat Fluid Flow 21(3):252–263. https://doi.org/10.1016/S0142-727X(00)00007-2 (**ISSN 0142-727X**)
70. Yildirim A, Kenway GKW, Mader CA, Martins JRRA (2019) A Jacobian-free approximate Newton-Krylov startup strategy for RANS simulations. J Comput Phys 397:108741. https://doi.org/10.1016/j.jcp.2019.06.018 (**ISSN 0021-9991**)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com